Analysis:

Problem identification:

Games are very popular In current times. Literally everyone plays or has played a form of single/online computer games. People of ages 5 to even 60+ play or have an interest in some form of gaming. The problem with games now however is that not all games can be played by the majority of users. This could be due to the sheer complexity of the game mechanics or the very time-consuming aspect of certain games. I understand that there are many different types of people out there each with different preferences and needs. Therefore, I have decided to create a game that is appealing to multiple users and be fun to play in one's free time. I will be doing this in python. I will also try to make sure my game is easily playable and does not involve too many complex mechanics. This is because I don't want to put off too many users from playing my game. There is also the problem of having to code complex mechanics as that for me as the programmer will be very time consuming and resource intensive.

The things needed in order to create this solution will be a stable computer that has the required software capabilities in order to run the game. There will also probably need to be Internet access in order for me to create the game and gather the resources needed. I will also need to have some form of abstraction in my game because otherwise it will make any updates or if people want to edit the code hard to do so.

Why my problem is solvable through computer science:

In order to solve my problem, the only possible way to do so would be through designing the code for the game. In order to design and make code I would require a suitable computer with an IDE just to start coding my solution. With all this I will then need to have a fairly strong foundation of computer science knowledge in order to code a solution that is suitable. I will also need to use other concepts from computer science such as abstraction when creating my solution. I will also need to use other aspects of computer science such as thinking procedurally and problem decomposition in order to break down my problem into small manageable tasks which makes it easier for me as a programmer to manage. I will be using an iterative development cycle when creating my game. This is because I will be progressively creating and developing my functions. At first it will be a simple and basic version of the game but over time it will get more complex and will soon be a suitable final product.

the use of Abstraction in my program:

While creating my game I will also use a lot of abstraction. Abstraction allows me to make my game look much more simple and to hide very complex aspects of it or by hiding irrelevant aspects of it from the user. For example I will need to hide any lines of code running in the background as the user plays the game. This is because people without decent knowledge in computer science will not be able to understand anything the code is

doing. This will only put them off playing my game and will influence the overall “liking” of my game. I will be using representational abstraction which helps me to simplify my game as I can view the entire problem and simplify the problem itself by only considering the key aspects of it. I will also be using data abstraction as within my code I will be using things like stacks and queues which all work in their own way yet I do not actually need to know myself exactly how they work. Abstraction also helps to reduce the time programmers spend.

thinking ahead when creating my program:

When creating my program I will need to think about the potential issues my clients may have when running my program. There could be many problems that a normal user may face when they try to play my game. By thinking ahead the developer can consider ways to solve these potential problems so that in the future it does not ruin the user's experience. The use of thinking ahead will help me make the game more overall easy and intuitive for the users as I can take into account their requirements and try to make the game tailored to their experience. Questionnaire for the stakeholder

Problem decomposition:

1. in order to create my program I will first need to think about how it will look and so I will need to create drafts of the design
2. I will then need to get the necessary IDE set up and ready
3. I will need to work out how to code my proposed design of the game
4. I will then need to test the code and check for any errors
5. I can then fine tune the code or make adjustments to improve the game Dividing into subtasks:

I will not try to code the whole program at one as that would be a big hassle and could end up provoking me into making mistakes. What I will do instead is that I will split the overall problem into smaller tasks that I could do individually. Each sub task will be a different function of the program or would be part of a function that I could do with some degree of effort. The smaller tasks when completed will be able to come together to make the program as a whole. By dividing my problem into sub tasks it will help me with scheduling and time management which is a very important aspect in creating a program since programmers have to work under the clock all the time. By breaking the problem into sub tasks it will also help me to avoid procrastination which is a big issue when it comes to being productive. I could set individual deadlines for each sub task and this will help me get through making my whole program quicker. If i make my full program slightly before the final deadline it gives me time to review and make and tweaks to it if needed

Stakeholders:

My games stakeholders will mainly be teenagers who are aged 14-19. In order to play my game pretty much anyone who is a teenager or older can play. This is because people of all age ranges who also own a computer will have easy access to my game and can experience the joy it brings. The reason my game will however be mainly teenagers from ages 14-19 is because this age range is when statistically people play the most games. This is because adults, even though they may still have an interest in games, may get caught up with work or other responsibilities and so they may simply not have the opportunity to play my game. I have also taken into consideration that teenagers also do have some degree of responsibility in their lives such as education or potentially things such as work experience etc. this is why I have decided to make my game a more fun oriented game rather than a straight story based. Time consuming game. I will however still try to include some aspects of a story because it makes the overall experience better. Another one of my stakeholders will be the people who have lots of free time and will be able to play my game without holding back. In order to satisfy these types of people the game needs to be somewhat challenging in order for them to stay hooked onto it. This Is why I have decided to make multiple levels that get progressively more difficult as you go higher. My other stakeholders may be people who are completely new to games and may be using my game as their first experience. This is why I may create a short instructions page which will outline the basic controls of the game. My other stakeholders may be the casual gamers that have previous experience gaming and are just looking for a new game to try. This is why I will attempt to make my game appealing to look at so that it attracts these types of users and interests them **GUI (graphical user interface):**

The graphical user interface is an important part of my game. My graphical user interface needs to be visually appealing in order for it to be appropriate. This will mean that I will have to include a range of colours in the game as a whole in order to avoid the game looking bland. I will also need to make the whole user interface simple to use and easily interactable. This means for example making it easy to adjust the game's settings or to actually start a game in general. This is needed because a confusing graphical user interface will put many users off playing my game. In order to achieve this, I will need to be able to create a good piece of code that follows a well thought out design pattern that I must deduce beforehand.

Questionnaire for the stakeholders:

I will be asking most of my stakeholders some questions related to my product. This will help me gain feedback and further improve the development of my game. Certain questions will be given to each stakeholder as they have different needs and wants so I will need to gather different bits of information in order to tailor my product to each of their needs. I will be conducting two questionnaires, one on an experienced gamer and the other on a more casual player. This is so I will get a range of opinions that can aid me in the games creation

Questionnaire for the casual gamer:

1. How many hours of games do you play a week?

2. What brought your interest into gaming?
3. What are your thoughts on a game that allows you to move around a designed map and kill monsters in order to level up?
4. Do you prefer games that you can play for a long time that are very objective/story based or do you prefer more quicker paced fun to play games?
5. What kind of colour scheme do you like in a game?
6. Do you have any examples of games you like and specifically what aspects of the game come to mind?
7. What kind of game genre would you recommend me to make?

Reasoning behind each question:

Question 1:

I am asking this question in order to find out how long this person plays in order to get an idea of how I can make sure my game is enjoyable during the time period my users play for

Question 2:

I am asking this question because I want to find out the reasoning behind why some people play games. This will help me to see the reasons for why people decided to get interested in gaming.

Question 3:

I am asking a casual gamer this question because I want to see what their opinion on the game I am planning to create is and so this will help me directly know whether or not these people like my game genre **Question 4:**

this question will just depend on what type of person I ask the question to but it will help me have a greater understanding of the type of game my consumers want

Question 5:

The colour scheme is a very important aspect in creating a game as that is the fundamental thing the users will be looking at. I asked this question in order to find out what kind of colours my users like in order to design the best-looking game for them **Question 6:**

This question will allow me to find out what type of games my users like and hopefully if they are able to pinpoint the exact aspects of the game I will be able to use that in order to develop my own game.

Question 7:

this question will help me decide on which genre my game as at the end of the day everyone has different preferences and so I will need to make sure I make the best decision when picking the genre **interview on jamie who is a casual gamer aged 19: question 1 - How many hours of games do you play a week?**

Response: “I play a total of around 4 hours of games a week and that’s split between the week mainly in the weekends when I am free”

Question 2 – what brought your interest into gaming?

Response: “ I had some free time and so a friend of mine encouraged me to start playing games as they are fun and a way of relieving stress when I am feeling down or straight up bored”

Question 3 - What are your thoughts on a game that allows you to move around a designed map and kill monsters in order to level up?

Response: “ I think it sounds like a cool concept and I would be down to play something like that”

Question 4 - Do you prefer games that you can play for a long time that are very objective/story based or do you prefer more quicker paced fun to play games?

Response: “I prefer to play games that are quicker to play and are fun oriented this is because I can’t afford to play too much just to get to the main climax of a game”

Question 5 - What kind of colour scheme do you like in a game?

Response: “ I like bright colourful games that are vibrant as these colours keep me attached to the game and are appealing to look at”

Question 6 - Do you have any examples of games you like and specifically what aspects of the game come to mind?

Response: “ I like to play games like fifa a lot. This is because i'm interested in football but the main aspect about the game which I like is that I can hop onto and play the game at any time and it doesn’t involve me having to keep playing and passing levels in order to have fun”

Question 7: What kind of game genre would you recommend me to make?

Response: “ im not too sure about how genres work but as long as it's fun I would not really mind”

Analysis of what jamie has said:

From what I have gathered out of what Jamie has told me I have come to understand many things. First of all he says that he plays around 4 hours of games split across the week. This could be due to him having other hobbies or just being busy. This means that I should have content that he can enjoy while he plays for the few hours he does. He also said to me that the reason he became interested in gaming is because his friends introduced it to him as something to do when bored. This means when I’m making my game I should make it

enjoyable as possible in order to make sure word spreads round and I gain popularity. Jamie said that he prefers games that he can play and have fun without having to play for many hours just to get to the climax. This links back to my predictions earlier that casual gamers will prefer to play something enjoyable rather than time consuming. This means I should make my game design more experience based. Jamie also stated that he prefers games that have a nice vibrant look and for it to be visually appealing when playing. I will take this into account when creating my game and designing it. Jamie finally stated that he doesn't have any specific genre preferences. This tells me that most casual gamers don't have a specific genre preference and so it will mean that while I am creating my game, I may not need to worry too much about the type of game it is. Overall, the responses I got from Jamie gave me great insight into what casual gamers like/dislike.

2nd questionnaire for a more experienced regular gamer:

I will be asking more regular gamers a few questions in order to understand what they want and to also compare with the responses from a casual gamer **Questionnaire:**

1. How many hours of games do you play a week?
2. What brought your interest into gaming and what keeps you into it?
3. What kind of colour scheme would you like with the game?
4. do you prefer long objective based games that have a long storyline or a less time consuming more fun oriented game
5. Would you prefer a game you play to contain long complex mechanics?
6. In the game would you like for me to implement different weapons?
7. Would you like me to include different pickable skins for the default character?
8. Do you think it would be worth it for me to make a tutorial if I were to make a simple game?

Reasoning for questions:

Question 1:

I chose this question to ask because I want to know how many hours an experienced person plays, and I want to compare it with the previous persons answers **Question 2:**

I want to find out why the person started playing games and for what reason they continue to do so

Question 3:

I want to find out the opinion about my colour scheme from a more experienced player so that I have a range of opinions as the colour scheme is a very significant aspect of creating my game

Question 4:

I want to find out what type of game this user likes to play. This will help me to create the best possible game for the multiple users who will play my game **Question 5:**

I want to see whether or not players like complex mechanics as I need to know this while creating my game

Question 6:

this is more of a side thing to the game but I wanted to find out if it was worth me creating cosmetics to the game or leaving it

question 7:

this also falls under the reasoning of question 6 and it's about whether or not it's a feature worth me implementing **question 8:**

I want to see whether an experienced gamer thinks it's worth me creating a simple game in order to see if it's worth allocating my time to.

Response from barry a 23-year-old regular gamer:

Question 1: How many hours of games do you play a week?

"I play around 20 hours of games throughout the week, I normally play more on the weekend then I do the weekdays"

Question 2: What brought your interest into gaming and what keeps you into it?

"The reason I came into gaming is because most of my mates' played games and so they suggested that I join them. I decided to buy a console and give it a try and I instantly knew this was for me. The way I can stay hooked to a game is through a game with a long engaging story that I feel obliged to complete.

Question 3: What kind of colour scheme would you like with the game?

"In the game I would like a colour scheme that matches the style of the game. for example a game that is meant to be scary should have a matching theme to the games style"

Question 4: do you prefer long objective based games that have a long storyline or a less time consuming more fun oriented game

"I prefer long objective based games that have a good storyline because it keeps me engaged in the game and allows me to keep playing it without getting bored of doing the same thing. For me it's not a problem as I play a lot anyway"

Question 5: would you prefer a game you play to contain long complex mechanics?

"I would like my game to have somewhat complex mechanics incorporated with it. This is because I don't want everything to be automated for me as that would be boring. I also want for the mechanics to not be too complicated however because I do want to focus on enjoying other aspects of the game.

Question 6: In the game would you like for me to implement different weapons?

“I think it would be a good idea as it would bring variety to the game and that will be a good thing”

Question 7: Would you like me to include different pickable skins for the default character?

“yes I think this would be a good idea as this goes under creating variety for the users”

Question 8: Do you think it would be worth it for me to make a tutorial if I were to make a simple game?

yes I think it would be a good idea as maybe for me it won't be such a problem but for others it could be very handy.

Analysis of what barry has said and comparison between previous responses:

Barry gave a good response to my questions and that has allowed me to have a good understanding of what people want from my game. Barry said that for the colour scheme he thinks that a colour scheme should be related to the theme of the game itself in order to be the best and most fitting. Barry had claimed that he likes the vibrant colours, and he wants the game to be visually appealing and so I will try to incorporate both these aspects into the game. Barry said that he enjoys long engaging story-based games whereas Barry wants a game that is easy and fun to play. In order to satisfy both of these groups I will need to consider both sides of creating my game. Barry had also said that he would like me to implement different weapons and skins for my game. This means that he likes to have cosmetic aspects in the game. This could be an important feature in my game as the users will like to have these features implemented. Barry also said that he doesn’t want the game to be automated and he wants some functionality in the game in order to not become bored. This means I will have to allow him to be in control of the game and not make everything too simple. He also said not to make it too complex which tells me that even experienced regular gamers do not want incredibly difficult mechanics. Barry claimed that implementing a tutorial may not benefit him but it may be necessary anyway for the newer gamers. Overall the two people I questioned had different opinions but they helped me understand what my clients wanted in a game

Research

In order to create the best possible game I will be researching by looking at current existing solutions for my game this will help me make my game to the best it can be. The research will also allow me to work out how and what I need to do to build my game.



Minecraft Dungeons on Steam (steampowered.com)this is where I got the image above from

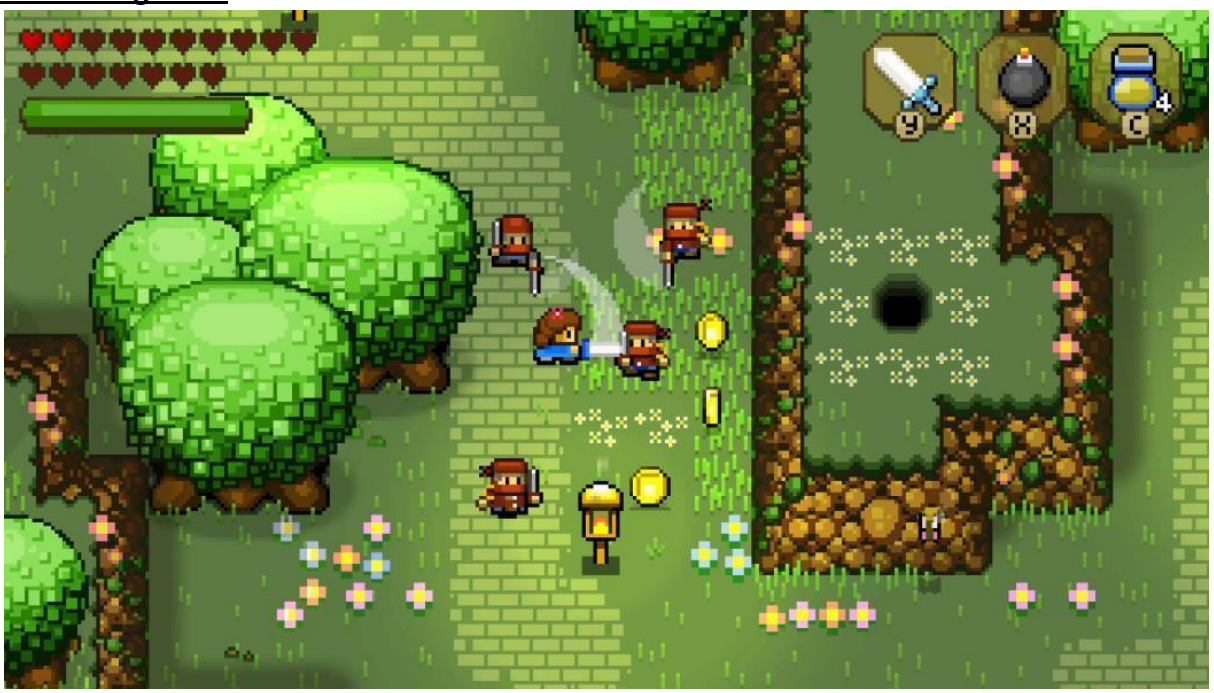
The first existing solution I decided to look at is Minecraft dungeons. This is currently the most popular dungeon crawler game out there so there was no surprise that I had to look into it. Minecraft dungeons is also one of the newer games made compared to other known dungeon crawlers. This game is different to other dungeon crawlers as the player gets to play in 3d view. Other dungeon crawlers are in 2d which could be the reason why minecrafts one stands on top. The game includes many features such as the ability of killing mobs or looting. This game also contains the use of using multiple weapons while progressing through the game. Players can also experience the game's storyline as this is a major factor in the games playability. This game also has good graphics that may only be fully utilised with a higher end pc or games console.

Bits I can consider implementing:

Minecraft dungeons is a very complex and high end game that I definitely cannot replicate fully due to limited resources and time. This game's 3rd person view aspect is an amazing feature that distinguishes the game from others but for me It may be far too complicated and ambitious to implement that, the games graphics are also very high quality but in my

game I will most likely include simple lower quality graphics. From the game I also believe that I can include the weapon changing aspect while progressing through the game. I will not be able to have as much variety but there should be more than 1.

The Zelda games:

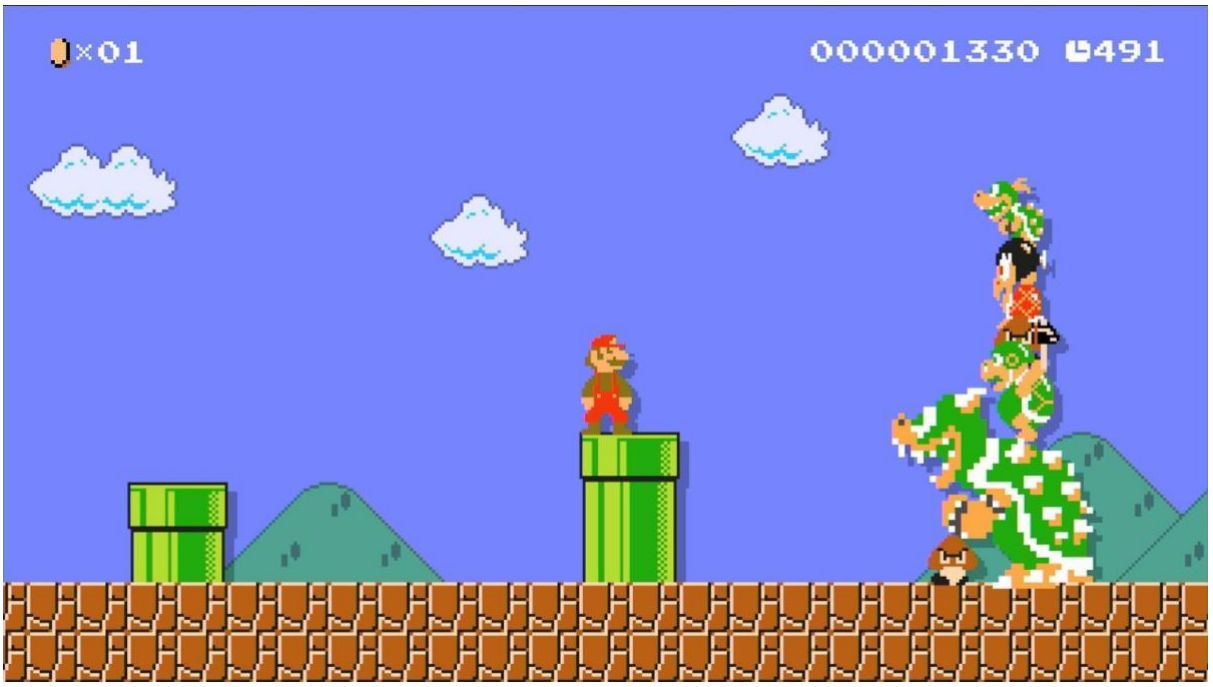


I got the image from [Play These "Zelda" Games on Nintendo Switch (pcmag.com)]

The Zelda games are a popular well known franchise of dungeon crawler games. The game is like your stereotypical dungeon crawler and is much more simple of a game then Minecraft dungeons. This game as the screenshot shows has significantly less quality graphics compared to more modern games but the game still has good enough graphics to be enjoyed and played. There is a health bar that shows the user how their character is doing. The game's mechanics are also very simple as the two main functions of the game Is to move your player and attack the enemies. There are also ways to interact with things such as coins and items. In the older games especially the story is told through a box filled with text that pops up when the user does certain things. **bits I can consider implementing:**

When creating my game I can include the aspect of Zelda and that being the game is very simple and easy to move around/play. This will make my game better suited for my intended purpose of it being a simple game to play. I will probably also consider adding a similar kind of health bar to my game as the user will obviously need to be aware of the health they have. I may also consider adding a similar sort of text pop up mechanism that would outline a basic storyline for the user

The older mario games:

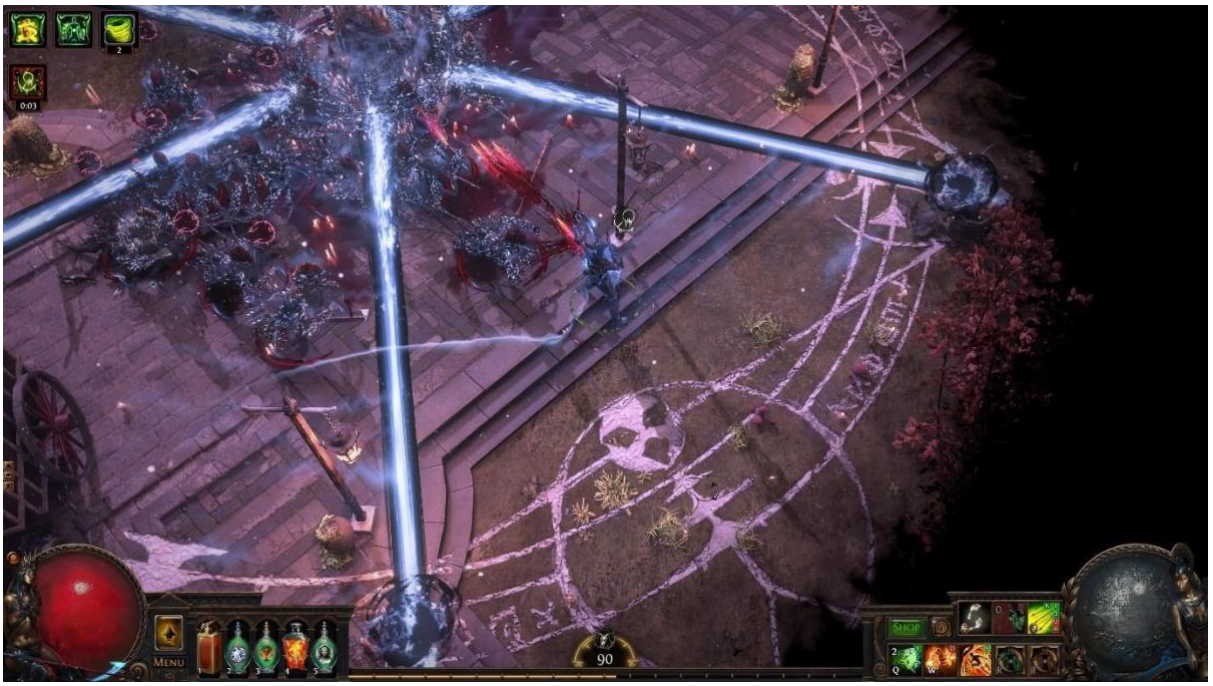


Super Mario Maker: An Ode to Old-School Challenge - GameRevolution this is where I got my image from

The super mario games are very well known by the community as they have been around for a long time and have been very popular. Super mario now does not actually look like the game shown in the screenshot above, this is because now over time they have improved the graphics significantly and the overall aesthetic of the game has improved a lot too. However, it used to be a good representation of what I am trying to achieve in the creation of my game. The older mario games consisted of simply moving along a 2D screen and avoiding getting hit by any of the monsters. The game also had a life based system where you had a set amount of lives and if they all get used then the game ends for the user. In mario there is also a system where you can gain coins which is one of the main objectives of the game aside from progressing through levels.

Parts I can apply to my solution:

In my solution I can take many of the aspects of the super Mario games into consideration. For example, the whole life-based system is appealing to me as opposed to a health-based system such as Zelda. I am not fully sure which one I will stick to, but it is an option for me to consider. I also like the whole coin-based system Mario has as this is sort of like an extra game objective players can aim to complete. This will allow my game to have more to it other than simply just being about moving around and killing monsters. From the Mario games I also considered the aspect of how there is a level progression system whereby after reaching the end of one level the user can decide whether they progress onwards. I can obviously not implement a lot of extra levels like the Mario games have but I can try to do like one or two.



I got this image from
https://www.google.com/url?sa=i&url=https%3A%2F%2Fwww.pcgamer.com%2Fpath-of-exile-hits-all-time-player-high-in-wake-of-latest-update%2F&psig=AOvVaw3DmDOKk4pkC8pSW-j32M_h&ust=1633039206822000&source=images&cd=vfe&ved=0CAcQjhxqFwoTCKCSx5-XpfMCFQAAAAAdAAAAABAE

Path of exile is a very advanced dungeon crawler and has a lot of functionality compared to other games of the same genre. For example this game has a number of selectable characters each with different abilities and it is the players job to select and master the correct character in order to progress through the game quickly. This game also has a skill attribute sort of system where you can choose certain aspects to upgrade that are from the skills tree. This system allows for even more variety in a game because it allows for the user to make specific builds of a character. This game also has a substantially high level of graphics compared to other games of the same genre which makes it stand out from the rest. The game world in a similar way to other dungeon games where you would have to go around killing monsters and progress through the chapters which consists of a storyline

How this helps me in creating my solution:

This is definitely a good game but there is not a lot I can add to my solution that I already do not have. This is because most of the things this game consists of are out of my capability to produce and may be too hard for me to code. For example I would like the idea of implementing a skills tree that allows the user to pick and choose what attribute the user wants but implementing something like that would be too expensive for them to do. From this game I also won't be able to adopt the graphic/visualisation as this would be incredibly difficult to develop.

Core features of my game:

The first thing that comes up when opening my game will be a screen showing the homepage and the options within the homepage. For example it may contain the start/resume button in order to begin playing the game. my game will then start with the player/character beginning in the middle of the room. There will be a health bar and a coin counter on the top of the screen. The player will be able to hold a weapon or something too. The player of the game can then move the in game character through the use of the arrow keys on the keyboard. This will move the in-game character to whatever direction desired. They will have the ability to move across the room and explore it however they wish to. The player could then either run into a mob that is killable, however if contact is made with the mob then the player will lose health and if done repeatedly then the player dies and the game automatically restarts with the character spawning at the beginning of the room. On the other hand If the player is successful in killing the mob their coin counter may go up. The player can then move on and more mobs may appear that need to be killed. For now my game may only consist of one room/level.

function:	**justification for it:**
home page	i will have a home page for my screen in order to make sure my users can play the game
a start game button that loads up the game	i will need a start game button on my home screen to allow for the users to be able to play the actual game
player that can interact with the map and can deal or take damage	my game will need to have a character that the player plays as. this would be the character that is shown in the centre of the screen and so it is obvious who the player is playing as.
mobs that deal damage	i will have mobs in my game that act as the enemy and these mobs can deal damage to the player and will act as the main threat within the game
coins that can be picked up	In my game there will be coins that could be collected and there will be a coin count on the top of the screen. the coins will be yellow

a end game screen	in my game there will be a screen that pops up when the player dies and it should be obvious for the user that the game has ended
background	I will have a background on my game. the map also counts as the background because the map is what the user will see once they are playing the game

Limitations:

There are many limitations in my game. The first limitation is that it will be very time consuming to code the game. This will be a problem as I can only allocate so much time to making the game and so I may not be able to make it to the best possible quality. Another limitation of the game is that it will involve me having to gather a lot of resources to actually create it. For example I would have to go through the hassle of having to get an IDE in order to make my game. Another limitation to my game is that many experienced gamers may find it boring after playing it for a while. This is because in terms of content and features my game is fairly simple. My game also will be a prototype. This means that I will not develop the whole game and may only develop a simple version of it. The limitation to this Is that players will be trying out an unfinished version of my game which may be buggy or not at its optimum level of fun. The reason I have decided to make my game only a prototype is because it will allow me to see how the whole game works and gradually build it up hopefully through feedback. The fact that my game is a prototype also means that when people may play it they could come up against a lot of bugs. This is bad for me because it could put up a bad impression of my game if they are unaware of the fact that it's in development. Another limitation to my game is that I will need to write efficient code that also doesn't comrise of too much memory. This is because if I want to distribute it to other people to play it then it will need to be easily stored and run. A limitation to the production of my game is that I will be working on it by myself. This is a problem as normally games are produced with a team but me being by myself means that it will take longer to create my project and it will be harder to create every little detail. On top of me being alone in creating this project I also have other courses I am doing and so this will mean I cannot dedicate all my time to this project. This means I will be under time constraints and this could hinder my development of the game. Along with the time constraints of making the game I will also have to consider the time taken to fix any unforeseen errors when creating my game. This is because there is a highly likely chance that my game will contain errors and bugs which I will need to account for and take time to correct.

Hardware requirements for the programmer:

A good computer: the programmer will need a computer with good specifications such as a fast processor, large amount of ram and up to date os systems. This is because when coding the app programming languages such as python will be used and in order for the programmer to do this in the most efficient way the computer needs to be as fast and usable as possible.

Mouse and keyboard: the programmer would need a mouse and keyboard in order to be able to utilise the computer

Software requirements for the programmer:

Windows/mac/linux OS: the programmer will need an up to date suitable OS system in order to create the game. The main three widely used softwares are mac windows and linux. This means it is ideal for the programmer to use these same software. The software should also be capable of running python as that's the programming language that will be used to develop the game.

Python ide: the programmer will need a suitable ide in order to be able to create and develop the program

Pygame library: in order to create the game the pygame library will need to be imported into the ide

Stakeholders requirements:

A pc to run the game: the user will need a suitable pc to be able to rub and play the game. it will need to have a decent processor as games involve processing a lot of tasks at once. The user will also need a suitable amount of cache and RAM because otherwise the computer would be too slow and would ruin the experience of the game

Keyboard and mouse: the user will need a keyboard and mouse in order to operate the laptop/computer and actually play the game

A nice simple layout: the users will need a simple design and layout in order to be able to quickly access what they want and be able to navigate with ease.

Stakeholder software requirements:

Suitable os: the user will need an os system that is compatible with the game. for e.g. they will need to have either of the three main ones which consists of windows mac and linux.

Success criteria:

criteria	justification for criteria
home page	I will need a homepage for my game containing the different opinions the user will have. this is need as it is one of my user requirements

suitable GUI	i will need a suitable graphical user interface as the users will need to be able to see what they're doing

option to start a game within home page	i will need this in the home page because users need to know what to press in order to play the game
option to open settings	i will need this option to open the settings menu within the homepage and within the game itself as i want to allow users to be given the chance to change their settings whether they are in game or not
option to pause mid game	i will need this option as the users may need to stop playing or access the settings menu randomly throughout the game and so it would be good to have a pause button
option to adjust sound	i will allow users to adjust the volume of the sounds as some people will have different preferences
option to change controls	people may prefer using their own specific controls so i will give them the option to do so in the settings.
simple design overall	I will need to make sure my game is simple as one of my user requirements is to make sure the game can be easily understood and playable.
the character can move within the game	i will need to know whether or not my character can move accordingly such as forward back left and right
the character has a animation while moving	i will need to make sure that there is an animation for the character moving whilst in the game
the character can collect coins/coin count	I will be including a coin system into my game, and I will need to know if my character can pick up the coins and if the entire thing works effectively
the characters health bar	i will be including a health bar as the player will need to know how much health their character has throughout the game

the character taking/dealing damage	i will need to make sure the game registers when the character gets hit or hits a mob
the map needs to be of a suitable size	the map cannot be too big or small as that would affect the users experience
mobs and character need suitable colour/looks	it needs to be obvious what the mobs look like in comparison to the character
mobs can attack players	The mobs can attack and deal damage to the players.
a score count	there will be a score count on the screen so the user will know how much score they have got
there should be an option to leave the game once paused	users may not wish to continue the game once it is paused so i would like to provide the option to leave
the game over screen once the the player dies	i will need the screen to clearly show the user when the game ends
the score system	the user will need to know whether or not their current score excesses their current score
option to return to homepage or start a fresh game	this is to allow the user to be able to easily choose what they want to do next

Design

my game will be a dungeon crawler type game. The game will be very easy to play and all the mechanics of the game are easy to understand. The basic outline of the game is that the player will be able to freely move around the map and attack the enemies. The user's main objective will be to try and get the highest score possible. if the players health reaches 0 the game over screen will be displayed and the user will be able to quit the application or play again

Aesthetic considerations:

- The game will need to take suitable dimensions and take a large proportion of the screen. The game will be 960 by 540 pixels
- The background should resemble a dungeon somewhat and look roguelike
- the player should also look like a character from a dungeon crawler
- the enemies should also have a design that looks intimidating and suiting of a dungeon crawler
- the coins will be relatively small and be around 10 x 10 pixels
- the colour of the coins will be yellow/gold
- the health bar will be a red or pink colour and contain hearts

I have chosen to make the coins yellow and the health bar red. This Is because the colour red for health bars and the colour yellow for coins are very traditional colours for these two things. This provides my users with familiarity as they will be used to seeing red represent health bars and coins yellow in based on previous game experience.

input aspects of the game:

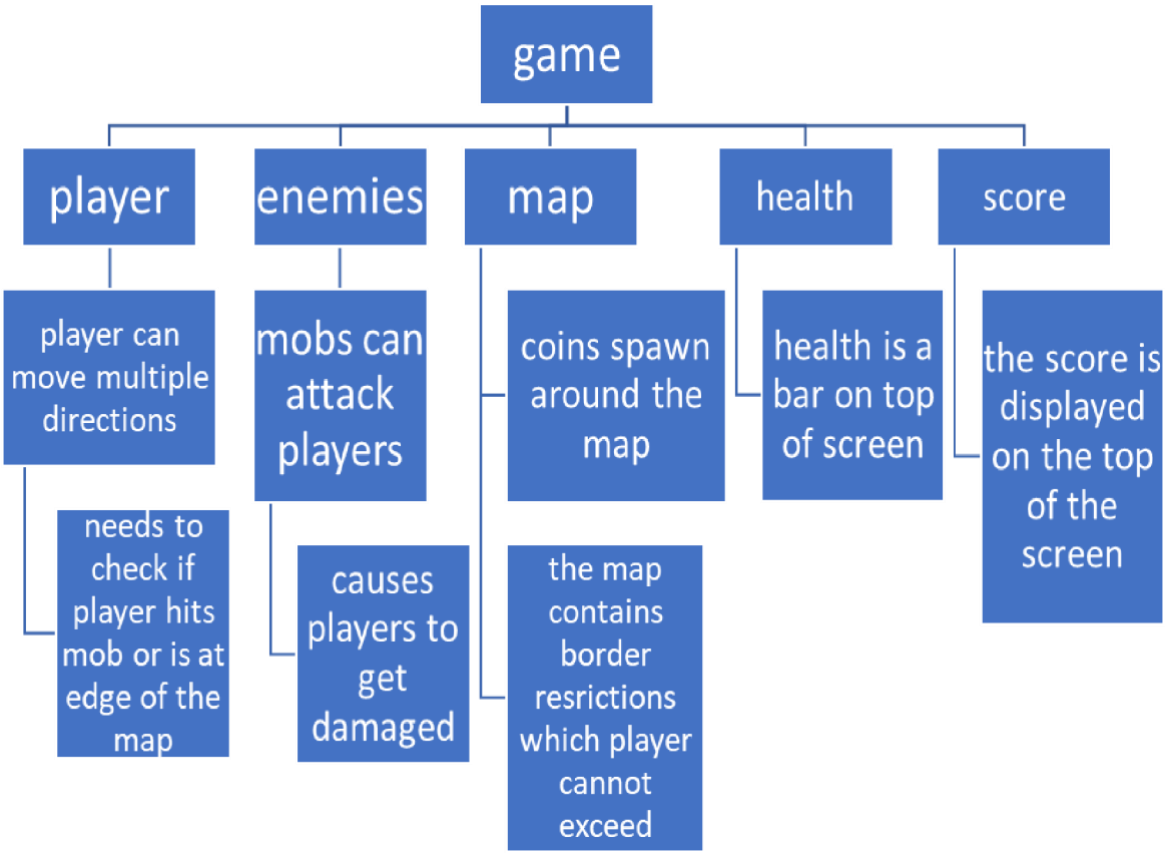
- the player will be able to select whether or not they want to go straight into the game or go on the settings menu or even go on the info section
- the player once in the game will be able to choose the direction the in-game character will move in
- they will then be able to move the character however they will ● the player can choose to then attack the enemy if encountered ● the player can then pause the game once they are already in it.
- Once paused the player can either check the settings or quit the game which if done will bring up a screen that says game over
- The player could potentially leave the game running of the pause screen assuming the game doesn't crash while the player is gone
-

Processing considerations:

- The game will need to constantly check where each inputted movement moves the character to
- If the player moves and hits a wall, then they should be restricted there and unable to move through a wall
- If the player makes, contact with the enemy assuming they haven't killed it by trying to attack it then a certain amount of hit points should be deducted from the characters health
- The game needs to also be able to tally up a score depending on the number of enemies killed and coins collected
- The game also needs to be able to show a game over screen once all the players health is taken away as in theory they are dead at that point
- The game also needs to make use of pathfinding algorithms to make the enemies find routes to the players position **Output considerations:**
- the score will need to be displayed on the screen to see how far the player has progressed
- the health bar needs to be displayed showing the health of the player on the screen
- the coin count and number of kills may also be displayed on the screen unless they contribute to the overall score as one thing

- the position of the player and the enemies need to be visually displayed on the screen for the user

top down design for being inside the actual game itself:



This diagram shows the overall problem represented by the heading game and it shows the different components it can be broken up into. the game can be broken into smaller sections that can each be all solved computationally. The game can be split into sections such as the player, enemies , map,health,score. Each of these represent key components the game will need in order for it to be able to sufficiently work. I have decided to use these because things like the player and the map are very significant components of the game and without these the game would not work or perform the intended function. **player section and link to success criteria:**

The player sectio represents the user/player side of the game. The player section can be further broken down into its components such as the ability for the player to be able to move in multiple directions. The player class will also need to be able to register when the player is at the edge of the map or hitting the mob. I have decided to do it like this because these functions that are individually part of the game act together to create the overall player class. This links to my success criteria because of the fact that I said I would have a suitable functional in-game player for the user to use.

enemies section and link to success criteria:

The enemies section will be a class that can be computationally solved in order to create the enemies. The class can be broken down into its components which are that the mobs can attack the player and cause the player to take damage. This is significant because the mobs in my game will need to be like this in order to be suitable to be classes as enemies. this links to my success criteria because the mobs in my game are meant to act like enemies

map and link to success criteria:

The map is a section that can be computationally solved for my game. The map can be further broken down into its components which are to spawn coins and contain right borders and also look appealing and suitable while also having necessary boundaries. This is important as once I create this it will come together to create my map overall. This links to my success criteria as I will need a suitable map for my game to function properly.

health and link to success criteria:

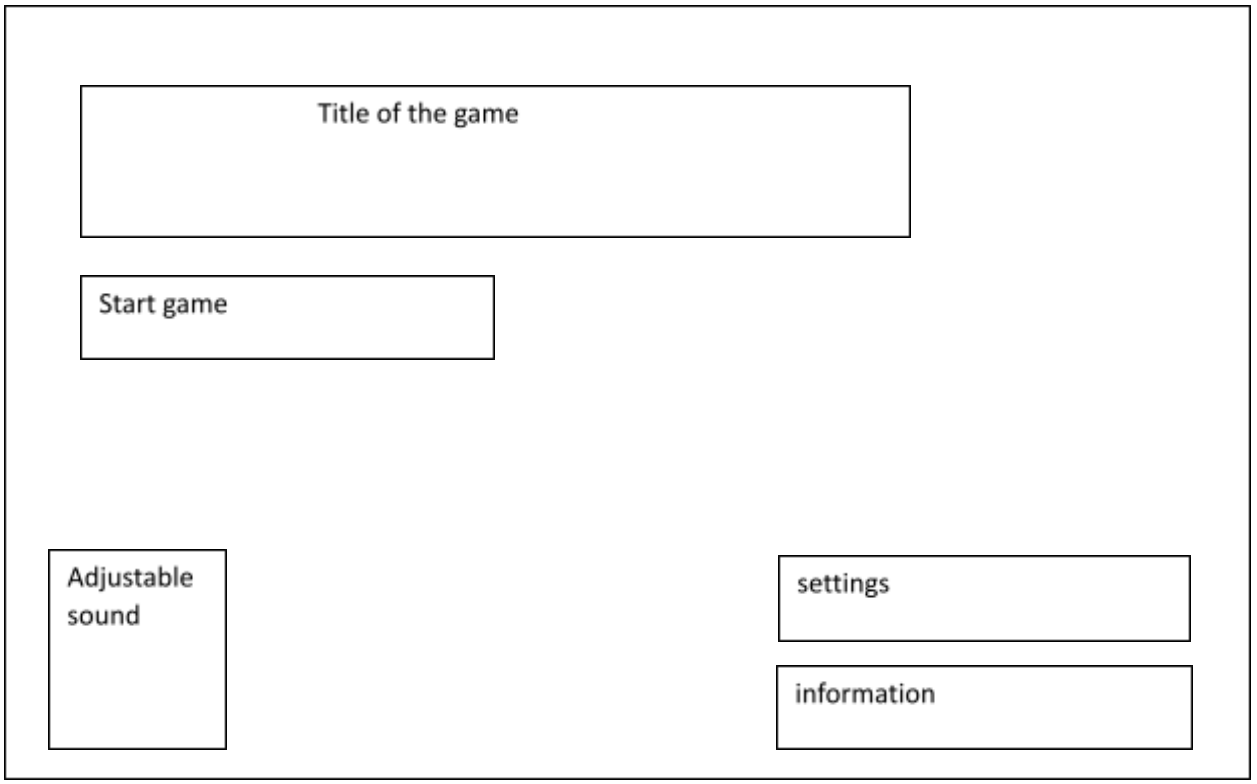
Health is a section in my game that can be computationally solved. The ehealth can be broken into its components which is to make sure the correct bits are shown on the players screen. The components can all come together in the health class to create health overall. This links to my success criteria as I stated the game will have a suitable health bar the user can base off.

score and the link to success criteria:

The score is a section of my program that can be solved computationally. The score is meant to show the amount of enemies the player has killed. The score is needed in my game and it can be made as a part of the player or health class/ it could even potentially have its own class. The score links to my success criteria as I said I would have a functioning score system on the top of my screen.

user interface:

When creating my user interface, I will be trying to make it simple and easy to use as one of my user requirements is that my game should not be too complicated especially for people who are new to the game. my interface won't have too many over the top features but it will contain the basic necessities a game needs.



have tried to make it look as simple and easy to navigate as possible to make it meet my stakeholder's requirements. I have included my games main options such as starting the game, the settings menu and the information section **Title of the game:**

This is used so that the user can see the title of the game and is more of an aesthetic thing as there isn't much functionality with it **Adjustable sound:**

This is a feature that's meant to aid the user and help improve their experience. The adjustable sound feature will be used to easily allow the user to choose a sound that's perfect for them and set the experience for them. I will try to make the sound a sort of bar/slider which can easily be dragged along for any changes rather than using number inputs for sound adjustments. **start game:**

The start game button will allow the user to start the game. the button will lead to the user being in the actual game screen and can then start to play however they like. The design for what the screen will look like once the user presses start will be shown later.

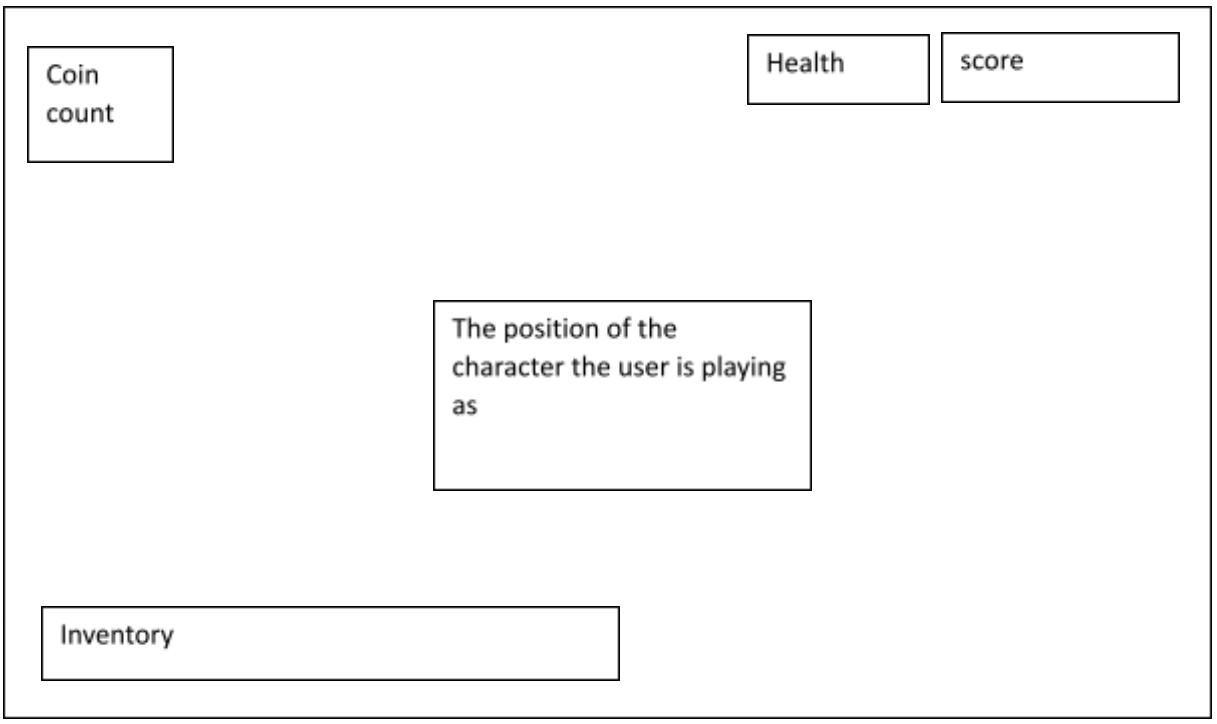
Settings:

The settings button once clicked will open up a menu that the user can interact with and can-do things such as adjusting key binds or even as a sensitivity function. This feature links to my success criteria as I needed a settings menu for it.

Information button:

This button will be used to show the user the basic controls of the game and the instructions to play the game. this feature is aimed to make the users experience a little bit better and it links back to my success criteria of making the game easily understandable

The heads up display (HUD):



coins are a little side thing the player can do while playing the game. Along with the coin count I may also include a picture of the in-game coins. This is because the user needs to know what that integer on the top left represents since a random integer may mean nothing to a new user. The coin image will be of yellow colour as that's the traditional colour for coins

Health bar:

The health bar will be a red coloured bar that represents the health points of the user. The health bar is going to be changing as the user will constantly be engaging with enemies so the amount of health will vary. I will use a heart image next to the health bar to represent the amount of health. This is because usually in games a red heart represents the amount of health a player has. This will allow my users to automatically associate the number next to the heart image to the amount of health the player has simply based off of familiarity.

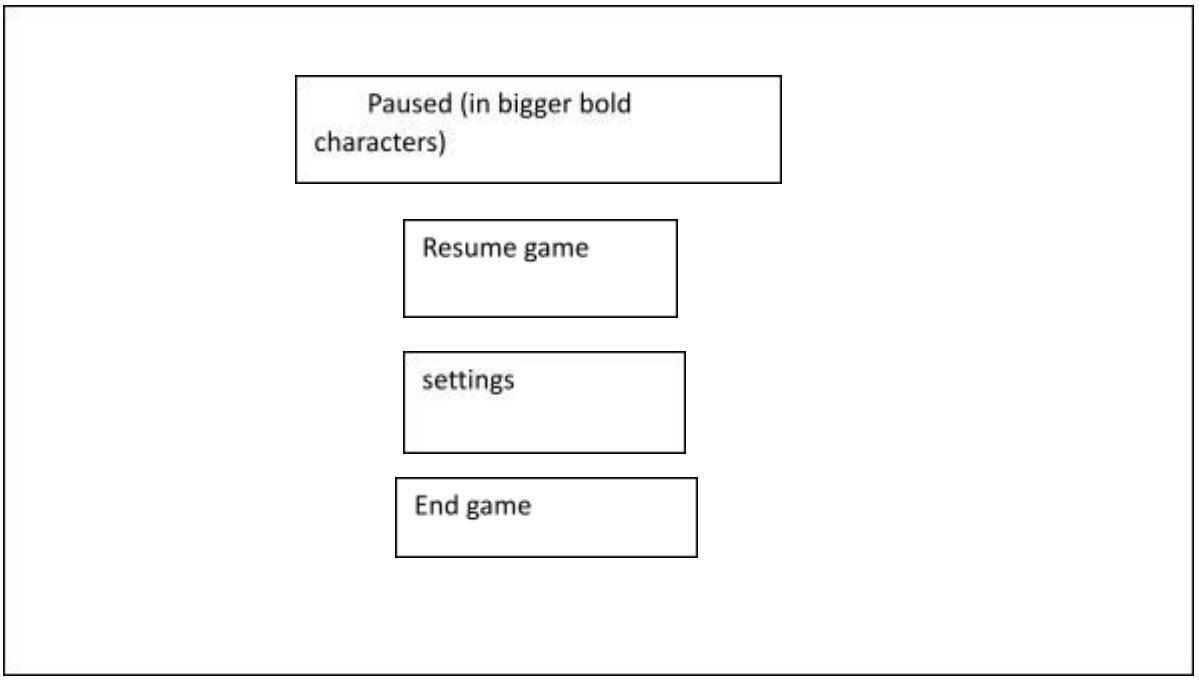
Score:

The score bar shows the amount of score the player has and it can be tracked as the player goes on. The score depends on the number of enemies the player has killed and the number of coins they have picked up. At the end of the game the score will be printed to the screen

along with the high score to show the player how they did in the game. **position of the character:**

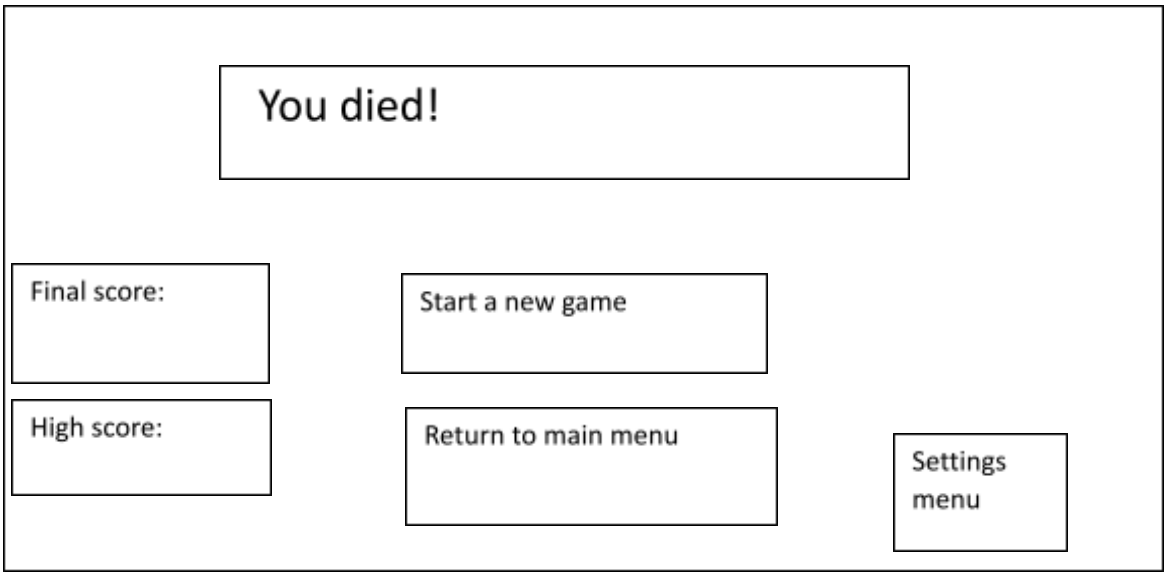
I have decided to position the character into the middle of the screen and have a third perspective view of the game. This is because I want the user's character to be the centre of attention as that's the thing the user is playing as and controlling.

The pause menu screen:



This is the screen that will be shown when the user decides to pause the game. The three main options are to resume the game, settings menu, and the end game screen. I have chosen to include a large text with the writing "paused" on the top of the screen in large and bold letters because the user needs to know what is currently going on. For example, if they leave the screen on and go to run some errands once they return to their computer, they will instantly know that they left the game on pause. The resume game and end game buttons are self-explanatory and do exactly what they look like they do. I added these two buttons to provide more functionality of the game and to give users the option to end the game early if they need to.

The end game screen:



This screen is designed to show what the screen will look like once the user dies. I had decided to make the main focus of the screen to be the “you died” text as in game this bit will be written in bold and will be also coloured red to resemble blood which links to a player dying in game. Aside from that there are options to straight away start a new game or to return to the main menu. This option gives players more functionality and flexibility with what they would like to do. In the left-hand side, I have included the final score and the high score of the player. This will allow players to see how they performed in the game they just played. I have purposely put the high score and score right next to each other because this will allow players to easily compare the two numbers with each other. Within the end game screen, I have also added a setting menu. This is because in case players would like to adjust their setting due to changes in preference they can do so easily without having to return all the way to the main menu to do it.



This is the background image I will be using for my game. The image is a suitable image since it resembles the sort of map u want. The map has darker colours on it which matches my game's style of play. The map will have boundaries on it but the player will be allowed to move in all directions. i got the image of the map from a free game assets website https://craftpix.net/



This will be the image for my enemy in the game. It is a suitable image as it resembles that of an enemy for a dungeon crawler and the user will easily be able to distinguish between the actual player sprites and the enemy sprites. i got the image of the enemy from a free website https://craftpix.net/



this will be the image for my player. This image is suitable for the player as it looks clearly like a character that is meant to take down the enemies. The character has a sword which will be used to kill the enemies and will be a crucial aspect of the attack animation aspect of my game. I got this image off a website for free. https://coderslegacy.com/python/pygame-rpg-player-class/



I will be using this as the health bar for my game. The design is suitable since it resembles a traditional health bar as it uses red hearts for the bar.

Stakeholder views:

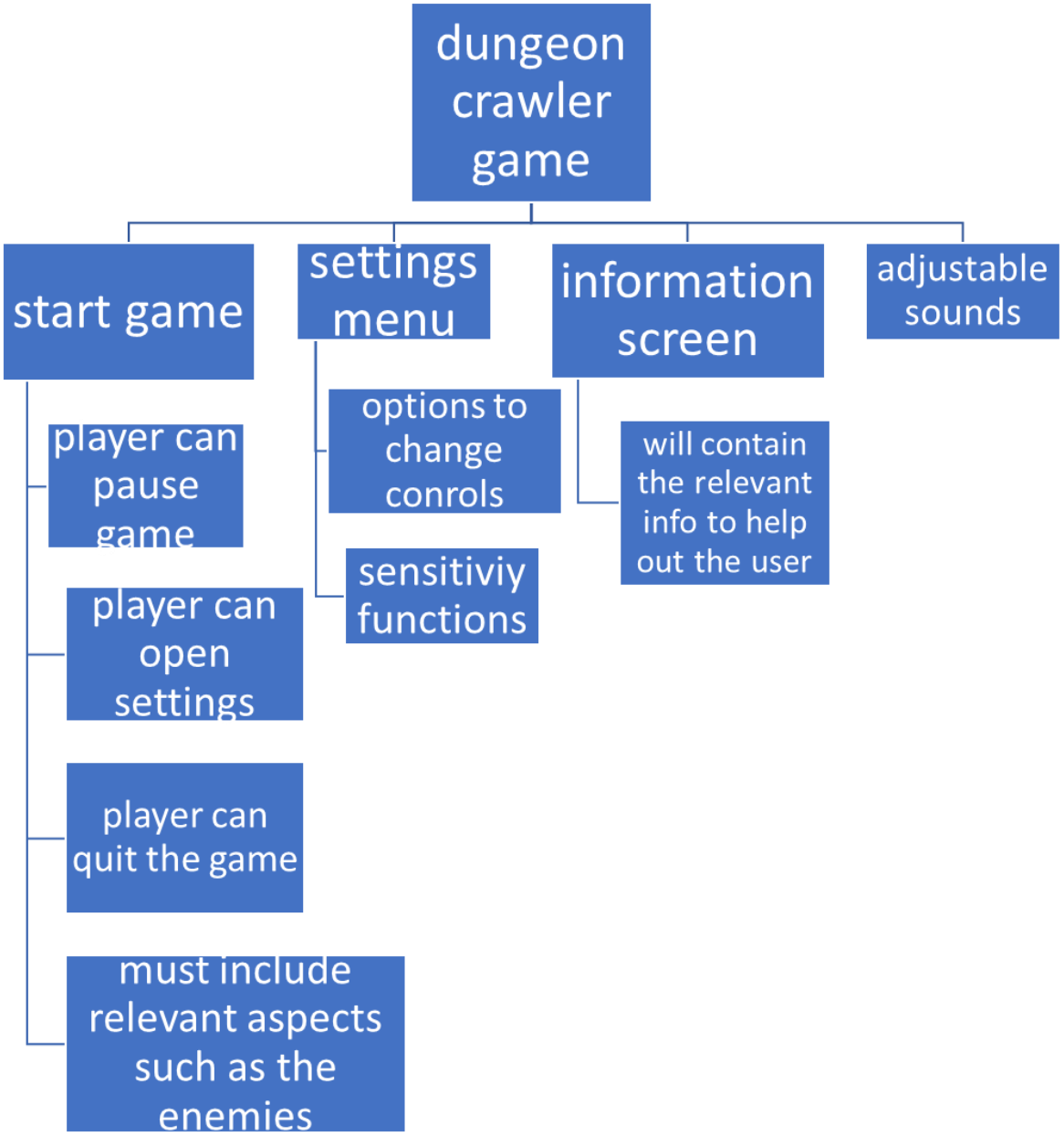
This is only a basic outline of what my game will look like but regardless I have decided to ask my stakeholders what they think about the design so far as their input is invaluable to the creation of my game **jamie**:

"I think the development of your game is going well so far from what I can see. The actual design looks simple and appealing. The easy to manoeuvre user interface is what I like most about your product as any new gamer can easily begin to understand how to play. Keep going at it, I think you're doing well!"

barry:

"The design of the game is coming together I see. So far it looks solid and if you carry on like this the main game should be amazing. I really like how you introduced a coin system. It really does add variety to the game"

top down design of the program as a whole



Algorithms for key functions of my program:

algorithm for the game loop:

while game is running:

check for any inputs # such as attacking or moving the player
constantly update contents being displayed onto the screen
check all events if pyagme.event.get = QUIT close game

This is the algorithm for the game loop. This is the loop that will keep the game running while the player plays the game. These steps are what the algorithm will do every frame of the game. This means that the game will be constantly refreshing/updating every second. this will continue to happen until the user willingly decides to end the game.

start game: procedure startGame()

score = 0 coinCount = 0 health = 100

gameLoop(health, score, countCount)

endprocedure

This will be the algorithm that is going to run when the game is starting. The algorithm initialises the variables for score, coin count and health. These are variables that will be shown on the user's screen as they play the game. This algorithm will also start the game loop which is a separate algorithm in itself and is responsible for making sure the game runs for however long the user is playing. The game loop will be a condition in which once it is met the game shuts down but this is depending on what the user decides to do.

adjusting key binds:

```
procedure changeKeyBinding(keyBindings:byRef, action:byValue)

key = input() if (key is already used in other action) print("key

already used") else keyBindings[action] = key

endif
endprocedure
```

This will be the algorithm I will use in order to change the key binds of my game. the algorithm will work by first checking whether or not the inputted key has been already binded to something. if this is the case then the user will be told that the key is already in use. this will then prompt a message for the user to pick another key. once the user picks a suitable key then the action the user chose will be binded to that key. this algorithm was implemented to make sure the user can have a quick and easy time changing the key binds.

algorithm for the player movement(horizontal direction):

```
procedure movement(self) self.velcoity = pygame.math.Vector2

#pygame variable keyboard_input = pygame.keys.get_pressed()

#pygame function if keyboard_input["left"] == True then

self.velcoity.x = -1 if keyboard_input["right"] == True then

self.velcoity.x = 1

endprocedure
```

This will be the algorithm for the player movement in the horizontal direction. it will take in a user input of either left or right. The key that determines whether the in-game character goes left or right will be dependent on their bindings. Once it is determined whether or not the user is going left or right the algorithm will then move the in-game character.

algorithm for the player movement(vertical direction):

```
procedure movement(self) self.velcoity = pygame.math.Vector2

#pygame variable keyboard_input = pygame.keys.get_pressed()

#pygame function if keyboard_input["down"] == True then

self.velcoity.y= -1 if keyboard_input["up"] == True then

self.velcoity.x = 1
endprocedure
```

This will be the algorithm for the player movement in the vertical direction. it will work in a similar way to the player movement algorithm for the horizontal direction. This means that for the character to move up or down the user will need to press the relevant key. Once this has been done the in game character will move positions.

Algorithm for coin count:

```
procedure coin() coin_count = 0      #this is the initial value once game

starts if player picks up coin, coin_count = count_count+1 else coin count

remains unchanged display coin_count to screen endprocedure
```

this algorithm will be the one responsible for updating the coin count once the player has picked up any coins. The coin count will be 0 at first and as the player progresses on and collects coins the number will change and this number will be displayed onto the screen.

algorithm for player health:

procedure health() player_health = 100 if player gets hit by

mob player_health = player_health sub 20

while health > 0
game does not end
endwhile display
player health
endprocedure

This algorithm will be used for adjusting the player's health. if the player gets hit by a mob within the game the player will lose a fifth of their health (20 hitpoints). this algorithm will recognise this and so will update and display the players health each time this happens. The while loop is there so that the player's health does not go below 0 as this would not make sense if it happened. This would mean that once the while loop condition is met the game should also be over.

algorithm for attacking:

playerAttack() attacking = false if pygame.event.type =

mousebutton attack = true enfunction

This algorithm will be used for when the player decides to do any sort of attacking motion. the game will set attacking to false initially and only when it detects the click of a mousebutton(or the relevant attack keybind) will attacking eb set to true. when attacking is set to true the player will perform the attacking animation and inflict damage onto the enemies.

algorithm for ending the game:

```
if event.type == pygame.quit # if the player decides to press the close button on top right

end pygame close down the application exit the system
```

This algorithm will be used for when the player wants to close the application of the game. they can do this buy either clicking the quit button or by pressing the x button on the top right that closes the window of the game

usability features of my solution :

My solution will have some usability features so that the users of my game can easily adapt to the style of my game without having to put too much effort into the game. For example, in my game I have decided to make it very easy to start up a game by making it so that the player only has to press the play button without the need to do any pre-game setting up. in my game i have also made it so that the player can easily quit the game by only needing to press one button

Throughout my game I have tried to make the whole look very familiar and memorable to users. I have done this by making it so that it is very clearly distinguishable between the player and the enemies. The enemies are green and zombie-like and this is a theme that users will be used to seeing. i have also decided to make it so that my health bar consists of red hearts which would make it easy for users to easily understand how that feature will work.

have made my user interface as a whole very simple and easy to use. This is through my simple design in which the user is presented with a few choices that then stem into other features of my solution. I have also decided to make the clickable buttons in my interface to be easily noticeable and accessible by the user so that it is that extra bit simpler. my user interface. i will be using bold letter in the writing of my user interface so that my users won't struggle to read the things on the screen

in order to maximise the levels of enjoyment for my users i have included some sort of customizability for my users. This is because in the settings menu the users can customise the sensitivity and the controls of the game. I could also potentially add things such as dark mode and adjusting the brightness of the game in order to make it so users can set up the game how they would like to. in my home page I have also given users the option to adjust the sound and put it to however loud they want it. This is because all people have different hearing abilities and so having one set sound level won't be ideal.

I will also try to include error handling within my game. This is so that if the user clicks the wrong button somehow produces an error there should be a pop up message in order to notify them of the error. This message should be user friendly and non technical so that the majority of users will be able to understand it.

Key variables:

Name:	Data type:	Description/justification:	validation and justification:
screen_height	integer	This variable will be used to set the screen's height and it will make it easy to use the variable across the program. it also gives the value context.	the screen height can only take integer values and the screen height will be set at start. This is to prevent random changes of the height variable and is necessary as many other aspects of the program will require the game height. there will also be a maximum height that the variable cannot exceed

screen_width	integer	this variable will be used to set the screen width of my games window. this variable will make it easy for me to access my screen width value across my code	The screen width also can only take integer values and it will be determined at the start with the height . it will be needed as the width is a necessary aspect throughout the program. there will also be a maximum width that the variable cannot exceed
playerHp	Integer	The player health with be shown in numerical form and if the player health reaches 0 it will cause the game to end	the player hp will be a variable that can only take integer values. There will be a maximum player health. there will be constant validation in order to check if the hp is less than or equal to 0 so that the game knows to end as the player has died
playerSpeed	float	This variable will be used to determine how fast the player moves on the user's	the player speed variable will be only an integer value and this will have to

		screen as they play the game. the speed will be higher than the speed of the mobs	checked/validated as only an integer will work

background	Pygame.surface	This is a pygame.surface object. It's used to show the game's background or colour.	there will be validation in order to check if the background image linked to the code is actually available in the same directory to be displayed
playerDamage	integer	the player damage variable will be a number and it will be an integer because integers are more appealing to the eye and also use less memory compared to the use of floats	the player damage will need to be an integer. there will be validation in place so that this value of damage in only inflicted when the player presses the mouse button
enemyDamage	integer	This variable will be used to calculate the amount of damage the enemies do to the player. it will be an integer so that less memory will be used compared to that of floats.	the enemy damage will be an integer and there will be validation in place to make sure this damage is only inflicted when the enemy hits the player
fps	integer	i will have a variable for the fps of the game and this will be useful for me when i want to use the fps throughout my code	
score_value	integer	This variable will be used to show what the player's score will be. this value will constantly be changing and be displayed while doing so.the score will increment by	there will be validation in place to make sure the score is an integer and cannot go below 0 as in my game there won't be negative score

		one each time an enemy is killed	
enemyHp	integer	This variable will be used for the enemy's health. if the enemy health is 0 then the enemy will die	there will be validation in palace to make sure the enemy gets killed once its hp is less than or equal to 0
enemySpeed	float	the enemy will have set speed and will be slower than the players so that it's possible to out manoeuvre the enemies as the user	
font	pygame.font	the font object will be used to assign a ttf to the variable and so this will allow me to use that specific font for when i display any text to the screen	the font will need to be validated to check if the file for the font exists in the same or correct directory

key data structures i will be using:

purpose:	type of structure	description/justification:	validation/justification:
enemySpawn	stack	the spawning of the enemies in my game could be through the use of a stack so that the first enemy in the stack will be	the stack will need to be validated to see if its empty or full and so no more enemies should be added to it
		the last to pop out	

enemySpawn	queue	the spawning of the enemies in my game could be through the user of a queue which would mean the first enemy in the queue will also be the first to spawn	the queue will need to be validated as if the queue is full no more enemies should be added to it and if its empty enemies should instead be added to it
attack and movement animations	array	I can use an array to index all the images that will be part of my animation. the images will then be shown one by one but quick enough so that the changes of the pictures will look like an animation to the human eye	there will need to be necessary validation in place to make sure the right animations array is being used for the intended direction only. so for example is the user moves right then only the array holding the images for the player moving right should be used
enemies traversing	trees/graph	The traversals of the enemies in my game can be done through using trees or graphs. the way they are used will depend on how the enemies intended to manoeuvre.	

associating player with the score	dictionaries/records	the use of dictionaries and records will man that in my game i can use the same structure in order to so the player name and they score they have regardless of the two being separate data types	validation will be needed to make sure that only the correct player receives an increase in score for completing an action and not all the players in the game

the use of classes in my program:

In my game I will be using classes in a number of different ways. classes help me by allowing me to encapsulate variables which means that outside the class the variables cannot be altered and so this will cause less problems for me while i make my code. Using classes I can also create multiple objects of the same class which can help me as I can simply use a constructor to make them rather than worry about.

class:	purpose and justification of the chosen class:
player class	I will be using the player class to create the actual on screen player that the user will act as. The player class will contain all the functionality of the player such as the movement and attacking. i will then create the player object which will allow the player to show up on the screen and execute all of the functionality within the class
enemy class	I will have a class for the enemies. The class will contain all the functionality of the enemies such as movement towards the player and the whole damaging aspect. i will then create many objects of the same enemy each time the enemies are to spawn.

button class	I will have a class for the clickable buttons on my game such as the play and quit buttons. The class will contain the button constructor and a method to check whether the button has been pressed or not. i will then create the objects of the buttons once i need want them to be available on the screen
health bar class	I will have a class for the health bar of my
	game. The class will contain the actual health bar images and the ability to render them. once i create the health bar object i can then make it so that the health bar will reflect the damage taken by the player
background class	I will have a class for the background of my game. the background class will contain my actual background png and i will be able to create an object of the background and blit it to my screen when i need it to show

test data to be used during development:

function being tested:	justification for it being tested:	expected outcome:	actual outcome of test:
game loads	i will need to know if the game will load up properly first of all	the game should load up correctly	
map dimensions are correct	i will need to check whether or not the map loads up correctly and fits the size of my games window correctly	the map should be the exact size of my game window	

player image and mob image have the correct dimensions	the player and the the mobs will need to have the correct proportion to the mao and each other	it should look fine to the users eyes	
mobs spawn and in correct amounts	the mobs will need to spawn in correct amounts and at places so the player does not get overwhelmed	mobs should spawn 1 every 5 seconds	
mobs should be able	the mobs are meant	should work	

to deal damage to player	to be able to deal 20% of the players damage per hit		
the health bar and score appear on the top of the screen	i will need to make sure that the health bar and the score loads up and is in their intended location on the screen	the health bar should be on the top left of the screen and the score bar should be on the top right of the screen	
the health bar adjusts when the player gets hit	i will need to make sure the health bar shows the correct amount of health the player has remaining	the health bar should reduce by one heart every time the player gets hit	
the score should correctly adjust	i will need to make sure that the score bar displays the correct score the player has	the score bar should increase by one each time and enemy is killed and it should be shown on the screen	

the player can move in the users intended way	i will need to make sure the in game player moves in the right direction the user inputs	player should end up moving in the intended way	
the movement animation works	i will need to make sure that the correct animations are displayed for when the player moves	movement animation should look correct	
the player cant exceed map boundary	i will need to make sure the player stays within the boundary	the in game player shouldnt be able to move outside the visible screen and be out of the users vision	
the player attack animation works	i will need to make sure the player attack animation works properly	the in game player animation should lick visually appealing and suitable	
when the player attacks a mob it should die	i will need to make sure that when the player attacks the mob it dies and gets off the screen	mob disappears after attack	
when the players health is 0 game ends	i will need to make sure that when the players health reaches 0 them game over screen shows up	game ends when player has no health	
player can quit or start a new game on game over screen	i will need to make sure the user can play again or quit the game on the game over screen	game over screen contains two options to play the game and quit	

testing post development:

what's being tested:	justification for testing it	result of test:
the entire game should run	i will need to make sure the entire program as a whole can run properly	
bug testing	I will be looking for bugs in my game. this is because my game may end up running but have some issues with it	
everything looks aesthetically pleasing	i will need to make sure that once everything has been created it should look good to the users	
can be run/opened on multiple different devices	i will need to make sure the game can be opened and run on a number of computers so that i know it will be suitable for my stakeholders	

usability and robustness testing post development:

After developing the game I will need to do some further testing to make sure that my game is good enough. I will first be testing the usability features to check whether or not the usability features of my game have been successfully met. I will need to make sure there are methods in place to make it easy for new users to understand the basics behind the game. I also need to make sure that my program is robust all around. This would mean that if the user was to input any incorrect data the program would be able to handle the request by either giving an error message or simply recognising it as a wrong input and not performing it with any specific functions.

development

initialising the variables:

```
import pygame
import sys
from pygame.locals import *
```

In order to begin the development of my game I had to import the necessary modules. i wanted to use python to develop my game as i specified in the previous sections and from what i learnt about python one of the best modules for developing my style of game was pygame. I therefore decided to import all of the pygame modules into my code so that i can use poygames built in functionality to aid in creating my game.

```
FPS = 60
FPS_CLOCK = pygame.time.Clock()
font = pygame.font.Font("square.ttf", 32)
# Global variables
SCREEN_WIDTH = 960
SCREEN_HEIGHT = 540

# Setting up color objects
BLUE = (0, 0, 255)
RED = (255, 0, 0)
GREEN = (0, 255, 0)
BLACK = (0, 0, 0)
WHITE = (255, 255, 255)
```

The next thing I decided to do was to initialise a few variables that I thought would be necessary to use. variables will help me to make my code more succine all throughout my code. I first decided to initialise FPS. I set the FPS to 60 as that's the normal value games use and it isn't too high so it should be suitable for most of my stakeholders devices. I then decided to initialise the screen width and height. I want the game to be more wide than high to mimic the nature of real computer screens. afterwards i decided to create variables for the most frequently used colours in general. I did this because python interpreters cannot understand the name of the colours but instead they work around RGB values. I decided to assign the correct RGB value to the colours I'm using so that when I want to use that specific

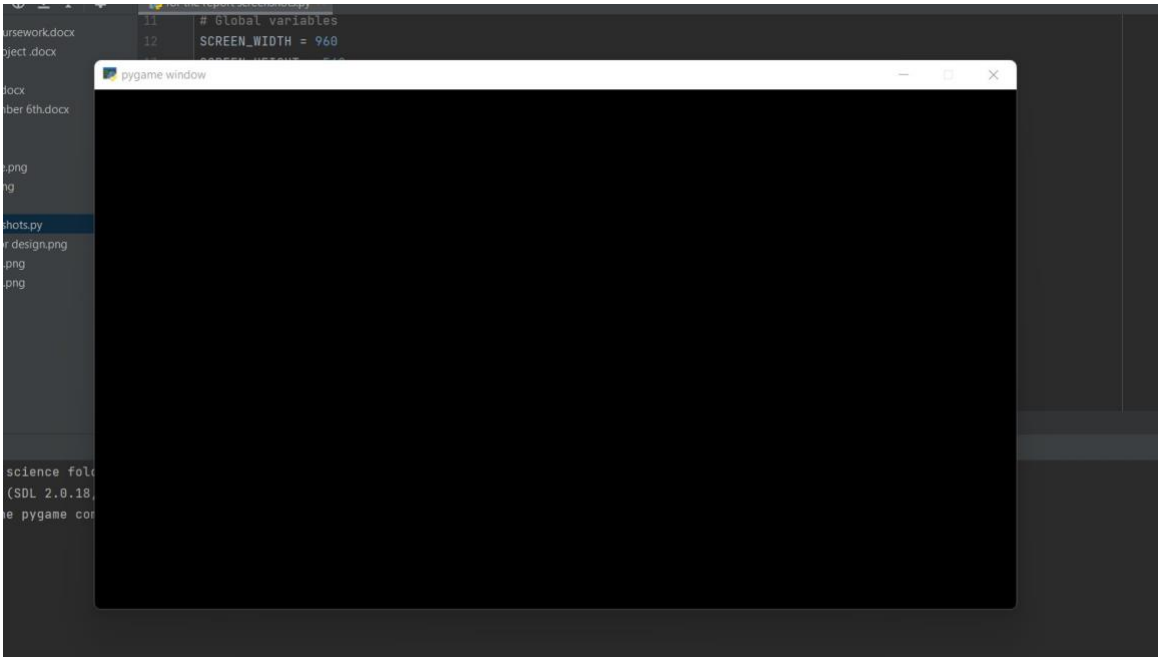
colour later in the code it becomes easy for me as the programmer to input them rather than having to enter the RGB values each time. I also have a variable for the font of my game. The font variable contains the actual ttf file of my font and the size of the font. This again makes it easier for me to use this font later on in the code.

```
pygame.init()
```

in order to use all of pygames functionality i had to initialise pygame itself right at the beginning.

```
# Set up pygame display object
displaysurface = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
```

I then decided to create a variable for the actual display of my game called displaysurface. The display is created through the already built in pygame function called pygame.display.set_mode which will help me create the screen without me actually having to do too much. I then pass in the necessary parameters which are the screen height and the screen width. The variables are equal to 960 and 540 which means that my display size will be 960x540 just like how I specified in earlier sections.



I ran the current code I have and as of now the only thing that comes up is a black screen. The black screen is the size I set on the screen width and screen height variables. this so far performed the intended function. I will then add a background with the same dimensions.

background:

```
class Background(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.bgimage = pygame.image.load("game_background_1.png")
        self.bgY = 0
        self.bgX = 0

    def render(self):
        displaysurface.blit(self.bgimage, (self.bgX, self.bgY))
```

I then decided to create the class for my game background. The class takes in the parameter pygame.sprite.sprite which is a pre-made pygame superclass that contains some already working methods I can use in my code if need be. i had to initialise the class at first and also initialise the superclass. I then created some attributes for the class. The attributes are class contained variables that can only be manipulated within the class. The self.bimage uses the pre existing pygame function pygame.image.load to load up my game image which is inside the brackets. I then created two variables called self.bgy and self.bgx in order for them to act as the x and y coordinates of the background image to be placed on.

I then created a render method for the background class. This method uses the preexisting pygame method “blit” to draw on the background image into my game window. it will draw the background on coordinates (0,0) due to the self.bgy and self.bgx parameters it took

```
# Initialise background object
background = Background()
```

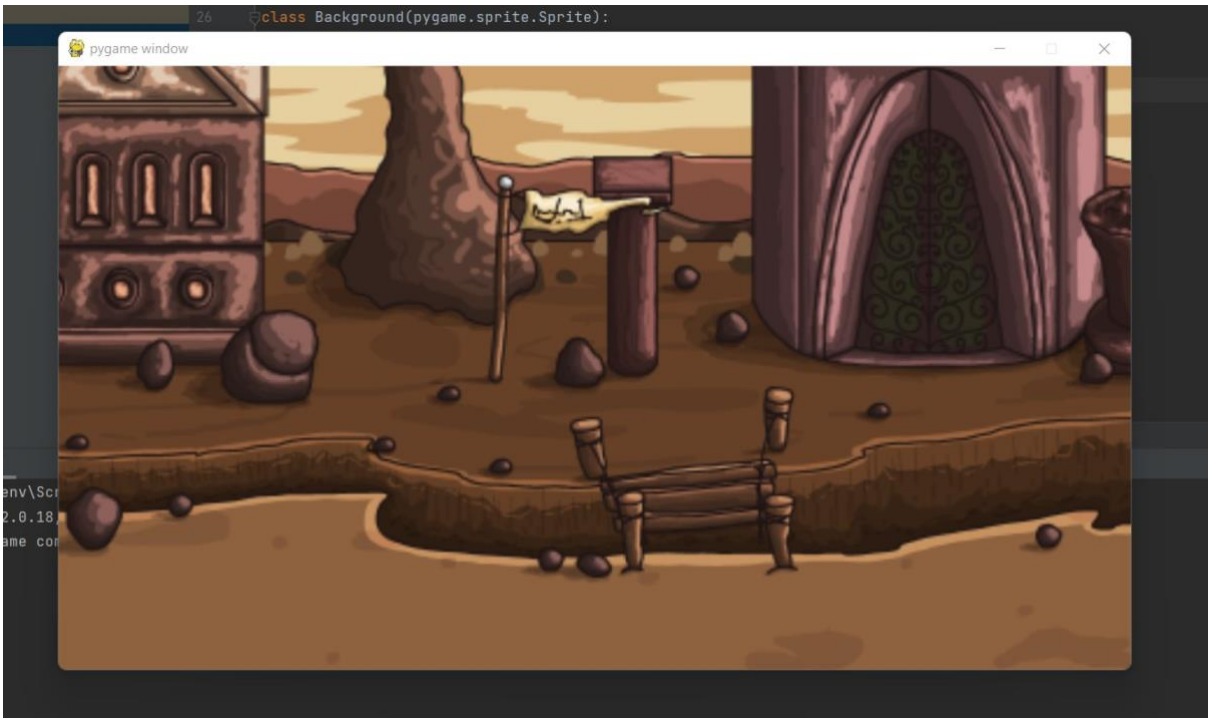
I then had to create the background object from the class so that the code actually takes into effect and produces a background for me.

```
# Game loop
while True:
    for event in pygame.event.get():
        if event.type == QUIT:
            pygame.quit()
            sys.exit()

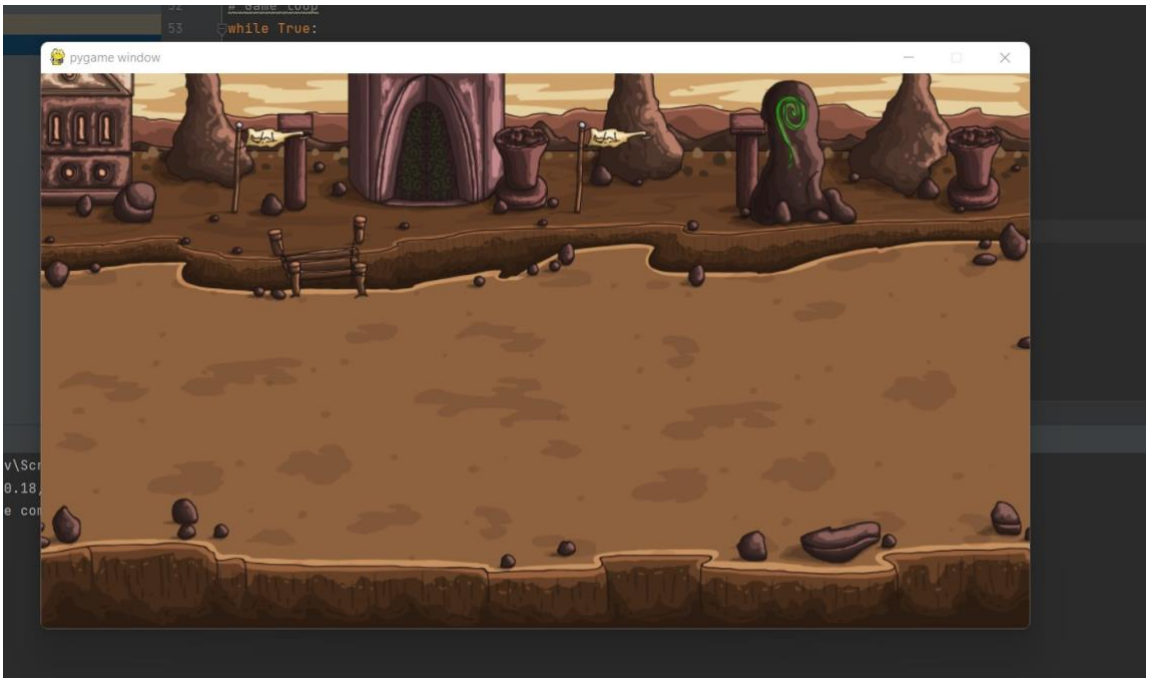

    # Render Functions ------
    background.render()
    pygame.display.update()
    FPS_CLOCK.tick(FPS)
```

I then decided that I needed to create a game loop. The game loop constantly checks for the pygame event QUIT. This is the action of the player clicking the close application on the top right of the screen. if this event is found to be true then the game ends and closes.

I also have a section in the game loop for the render functions. These consist of the background.render() which constantly renders the background object. There is also a pygame.display.update and fps_clocl.tick(fps) functions.



This is what happened when I first tried to run the code. The background game was way too zoomed in and it was not how I wanted it to look.



I then resized the actual image itself to match the exact dimensions of my pygame window. after doing that the background image fit perfectly into my screen.

after making the background i decided that it was now time to move onto creating the actual player of my game

```
class Player(pygame.sprite.Sprite):
    def __init__(self):
        # Inherit from the pygame.sprite.Sprite superclass
        super().__init__()
        self.image = run_ani_R[0]
        self.rect = self.image.get_rect()
        self.rect.center = (SCREEN_WIDTH / 2, SCREEN_HEIGHT / 2)
        # Combat

        self.attacking = False
        self.cooldown = False
        self.attack_frame = 0
        # Set the default variables
        self.health = 5
        self.score = 0
        self.direction = "RIGHT"
```

Player › __init__()

I started to create the player class first by initialising it and passing in the pygame.sprite.sprite superclass which allows me to utilise some of the existing functionality in the superclass for my code. I then started to create the attributes of the player class. i

created a variable called self.image which will contain the png image of the player but as of now it consists of an array for future use. I then used the self.rect function to make the game recognise the player as a rectangle with dimensions. This will help me to add in movement and collision detection later on as I make the game.

i then made a variable called self.rect.center which will be used by me to determine the starting position of the player.i wanted the player to be positioned onto the centre of the screen but instead of having to put in the coordinates of the centre of the screen i simply made it equal to the screen height and width variables divided by 2.

I then also made some other attributes such as health,score and attacking_frame. These variables are inside my class and as of now have no use but will become of use to me as i add more functionality to the player class.

```
player = Player()
```

I then had to create the actual player object from using the class template. This line of code has to be anywhere outside the actual player class itself.

i then had to make it so that the actual player is drawn onto the screen

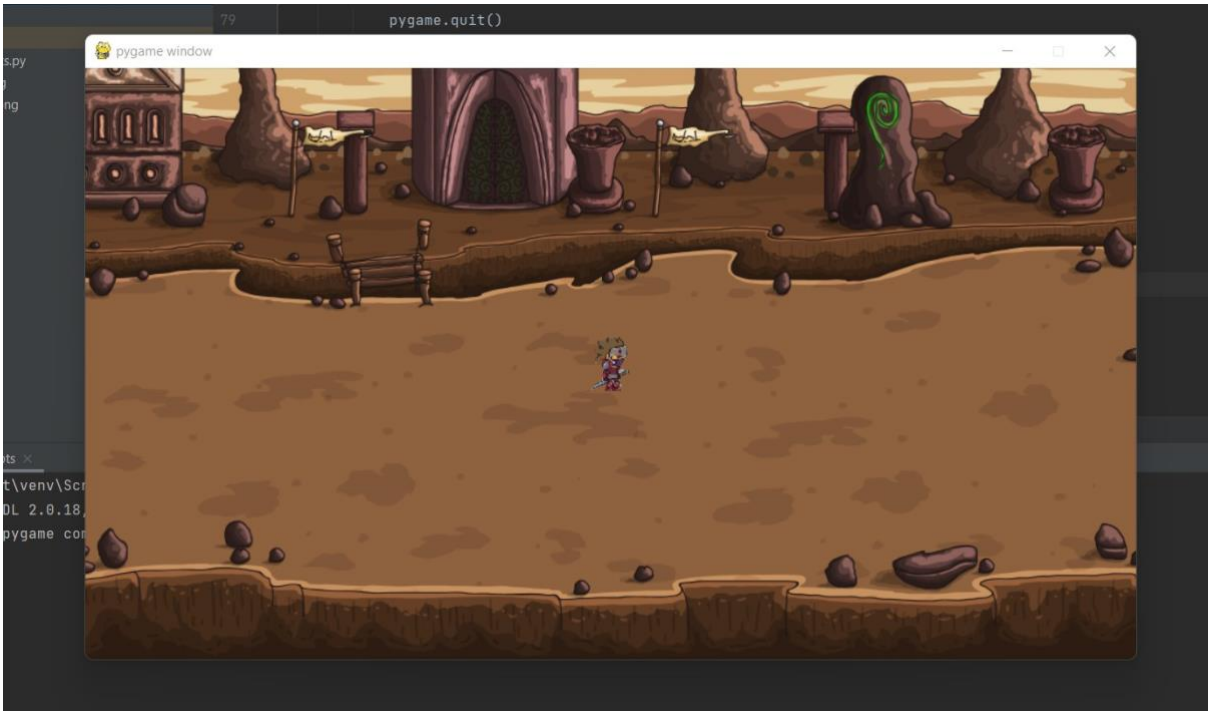
```
def draw(self, surface):
    surface.blit(self.image, self.rect)
```

I made a new method inside the player class called draw. This method takes in the image and the rectangle of the image from the class variables and then uses the pygame built-in surface.blit method to actually draw the player onto the screen.

```
# Render Functions ------
background.render()
displaysurface.blit(Player.image, Player.rect)
pygame.display.update()
FPS_CLOCK.tick(FPS)
```

next i had to actually draw the player onto my game using the displaysurface.blit method. this method takes in the player image and the player rectangle and then blits it to the screen. at first first i had issues because my actual player did manage to render onto the screen but the background image was rendering on top of the player causing it to not be visible on the screen. i then sorted out by fixing the order of the the lines of code inside of

the loop. i then made sure the pygame.display.update was after the render functions so that the code works



this is the result of what happens when my code comes up

review of what has been done so far:

So far I have mainly created some of the background and visual aspects of my game. As of now the only thing my game can do is display a player onto a background image of mine. Currently the game production is going good and I'm managing to get some of it done. I did run into some problems while making this part of the game but I managed to sort it out after multiple attempts. **how i tested everything:**

I tested the code by constantly changing the lines of code, the values of the variables and the order of the code. I did this. I always checked how the code works by running the code each time I changed something. I kept doing this till the code performed the intended job and loaded up the background with a player on it. **how the games meets with the stakeholders,design and success criteria:**

so far the game meets my design plans. I used the exact background and player images as I specified in my design. I also made sure I stuck to using images that match the theme of my game. I have also met some stakeholder requirements and success criteria requirements such as making the game look simple and easy to play. it is too early however to fully assess how the development of my game meets the overall requirements.

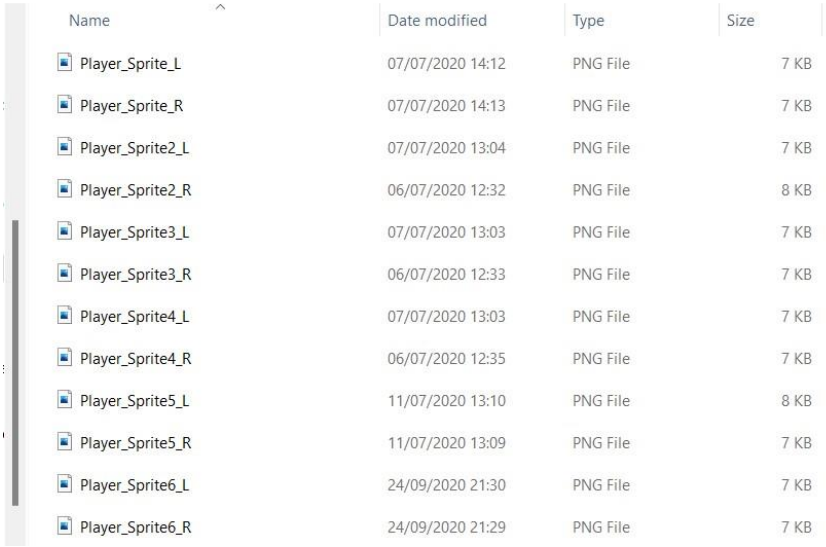
adding movement to my player:

After making the player I decided the next steps would be to add movement to the player. I have decided to make it so that the movement is based around the W,A,S,D keys.

As I was going through the process of implementing movement in my game I had to work out how to do a movement animation for the player moving inside of the game. pygame did not have any way of actually animating the player movement so I had to do it a different way.



In order to create my movement animation I had to use multiple images of the same character but each representing a different stage of movement. I then cycle through these images per frame increase in the game. this then creates an animation as even though its just different images being cycled through per frame for the human eye it looks like the player had fairly fluid movement



Name	Date modified	Type	Size
Player_Sprite_L	07/07/2020 14:12	PNG File	7 KB
Player_Sprite_R	07/07/2020 14:13	PNG File	7 KB
Player_Sprite2_L	07/07/2020 13:04	PNG File	7 KB
Player_Sprite2_R	06/07/2020 12:32	PNG File	8 KB
Player_Sprite3_L	07/07/2020 13:03	PNG File	7 KB
Player_Sprite3_R	06/07/2020 12:33	PNG File	7 KB
Player_Sprite4_L	07/07/2020 13:03	PNG File	7 KB
Player_Sprite4_R	06/07/2020 12:35	PNG File	7 KB
Player_Sprite5_L	11/07/2020 13:10	PNG File	8 KB
Player_Sprite5_R	11/07/2020 13:09	PNG File	7 KB
Player_Sprite6_L	24/09/2020 21:30	PNG File	7 KB
Player_Sprite6_R	24/09/2020 21:29	PNG File	7 KB

inside the movement animation folder there are all these images that i will use to create the players movement animation.

```
run_ani_R = [pygame.image.load("player/Movement_Animations/Player_Sprite_R.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite2_R.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite3_R.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite4_R.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite5_R.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite6_R.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite_R.png")]

run_ani_L = [pygame.image.load("player/Movement_Animations/Player_Sprite_L.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite2_L.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite3_L.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite4_L.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite5_L.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite6_L.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite_L.png")]
```

Before I can use the images inside my code I decided to first create two variables, one being used for leftward motion and the other for rightward motion. Each variable uses the array data structure to hold the actual images of my player method.

```
def move(self):
    # Change condition values to fit in the background image setting boundary for the player to move
    pressed_keys = pygame.key.get_pressed()
    if self.rect.left > 0:
        if pressed_keys[K_a]:
            self.image = run_ani_L[self.move_frame]
            self.rect.move_ip(-5, 0)
            self.direction = "LEFT"
    if self.rect.right < SCREEN_WIDTH:
        if pressed_keys[K_d]:
            self.image = run_ani_R[self.move_frame]
            self.rect.move_ip(5, 0)
            self.direction = "RIGHT"
    if self.rect.top > 40:
        if pressed_keys[K_w]:
            self.rect.move_ip(0, -5)
            self.direction = "UP"
    if self.rect.bottom < 485:
        if pressed_keys[K_s]:
            self.rect.move_ip(0, 5)
            self.direction = "DOWN"
```

This is the actual movement method of the player class. I first created a variable called pressed_keys and assigned it to the pygame function pygame.key.get_pressed(). This made it possible for my code to check what key the user has pressed.

I then started by setting the boundaries for the map.for each direction i created if statements which only allows the player to move in that certain direction only if the statement is met. This allows me to make sure that my users cannot exceed the map even if they wished to or accidentally did so because the if statements will not allow the movement to go thorugh.

I then set the keys for each direction of movement in my game. For example, for upwards movement the W key is used and for rightwards movement the D key is used. for each

direction I then set the players centre coordinates to be 5 more in the respective direction. I then assigned each direction to the string version of its name.

The leftwards movement direction for the player's direction also contains a variable called self.image which is assigned to the array with the directories to the necessary images for movement. As of now this means that there is only an animation for the player moving left or right but not up and down. this is due to the fact that i did not have the required images to do so

```
if pressed_keys[K_s] == False and pressed_keys[K_w] == False and pressed_keys[K_d] == False and pressed_keys[
    K_a] == False:
    self.move_frame = 0
    if self.direction == "LEFT":
        self.image = run_ani_L[self.move_frame]
    else:
        self.image = run_ani_R[self.move_frame]
else:
    self.move_frame += 1
```

After making the player movement method I realised that the code still did not work properly. This was because the game did not know if the player stopped pressing the same key or whether or not they changed keys for movement. This is why I decided to make sure that the program goes through a sort of validation process to check whether or not any keys are being pressed. This code shows that if no movement keys are being pressed the frame resets to 0 and then depending on the direction the necessary images in the array are displayed per frame.

After testing this now my game started to crash after a few seconds of movement. This was because after some time the game kept on incrementing the frames and by doing so it was using the images one after the other. however i only had 6/7 images inside of my array so when the game tried to retrieve an image greater than 6/7 it could not handle the process as no such thing existed therefore crashing my game

```
def update(self):
    # Return to base frame if at end of movement sequence
    if self.move_frame > 6:
        self.move_frame = 0
        return
```

In order to correct the error I made an update method for my player class. this method automatically resets the self.move_frame value to 0 if any value above 6 is acquired. This will make it so that the same cycle of movement images are constantly repeated over and over to show the movement animation.

```
# Game loop
while True:
    for event in pygame.event.get():
        if event.type == QUIT:
            pygame.quit()
            sys.exit()

    player.update()

    player.move()
    player.draw(displaysurface)
    background.render()
    # Render Functions ------

    FPS_CLOCK.tick(FPS)
    pygame.display.update()
```

After creating the move method and the update method I decided it was time I could actually implement the movement. In order to do this I put the move and the update functions into the game loop. The methods inside of the game loop are constantly checked and are executed every frame. This would mean that every frame that goes in the game will check for movement and will output it onto the screen. I then also used the draw method from earlier on that I made when making the player at first. This method will be used by me to draw the player onto the screen per frame.

After trying to run the code I came across a few problems. for some reason the player no longer showed onto the screen at all. After trying it and testing it multiple times by changing around the code I came to the conclusion that the reason behind my errors was that the background was being rendered onto the screen after the player moved,updated and draw methods. In order to solve this I made sure that within the game loop the background gets rendered first before the actual player functions get called.

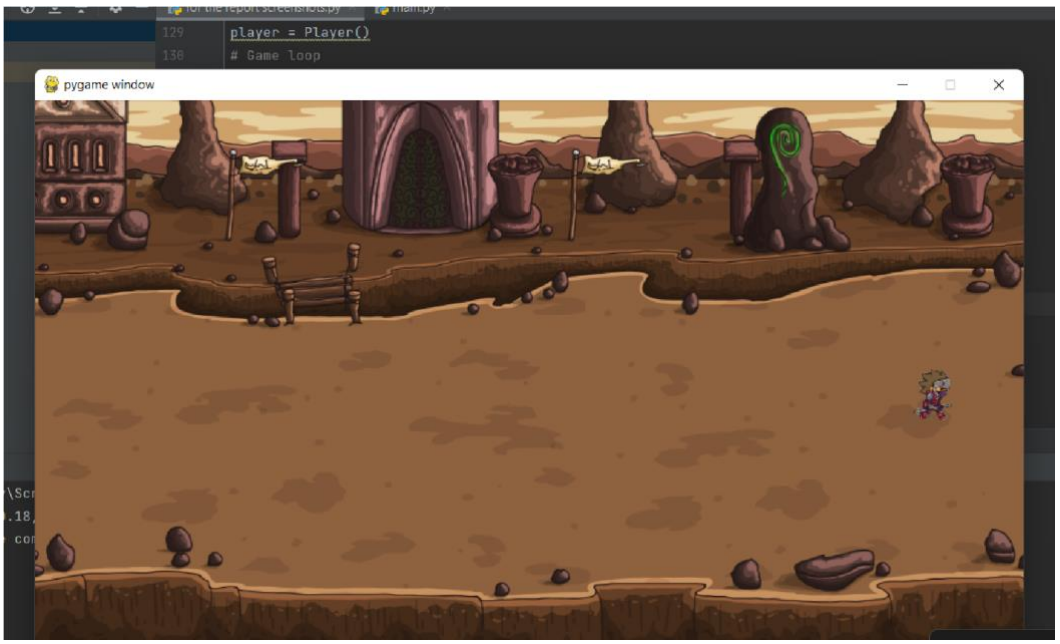
```
# Game loop
while True:
    for event in pygame.event.get():
        if event.type == QUIT:
            pygame.quit()
            sys.exit()
    background.render()
    player.update()

    player.move()
    player.draw(displaysurface)

    # Render Functions ------

    FPS_CLOCK.tick(FPS)
    pygame.display.update()
```

this is the newly updated game loop and after running it the game now works and the movement functionality of the game is working as intended **player moving right:**



This is an image for the player moving in the rightwards direction. The best way to demonstrate the player movement was to grab a screenshot of the player moving mid frame. i have also done the same thing to show the player moving in all the other directions

player moving left:



player moving down:



for the player moving downwards i do not have the necessary images currently to be able to create an animation for it. this means that as of now for moving down the player just faces right as the default frame drops across the screen

player moving up:



This is how the screen will look if the player moves up. i also did not have any pictures for the frames of the player moving upwards therefore i had to just leave it like this with the player facing right as the default frame moves up the screen.

review of the player movement development stage:

As of now I have managed to create a suitable movement mechanic for my game. The player can move in all 4 directions and there is a suitable border on the map in my game. I have also managed to create a fairly decent movement animation for the left and right direction of movement. This was done through using an array data structure to cycle through different images of the player in order to create the animation. **how i tested everything:**

In order to test the movement code I was constantly changing around the statements and the pieces of code while running the code here and there to see the results. I then managed to get some suitable code for the movement that ran pretty fine. After testing the code I decided to then test the actual ingame movement of the player as if I was the user. I tried to press random keys on my keyboard just to see what would happen if I did so. I then came to the conclusion that the only buttons that caused the player to move were W,A,S,D which is the intended method. **how the section meets with the stakeholders,design and success criteria:**

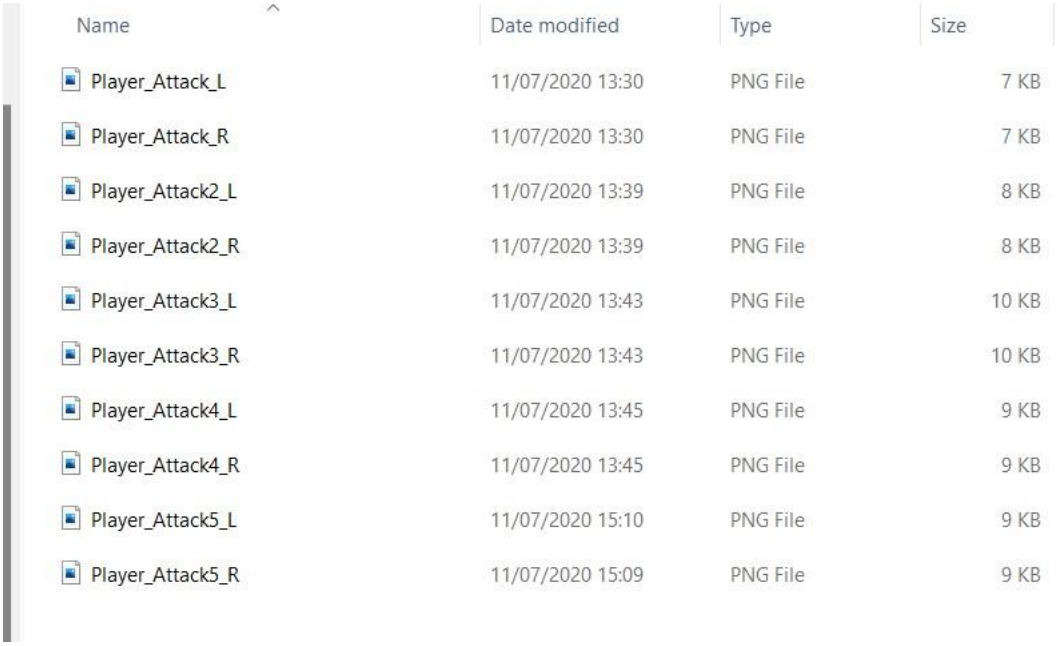
The actual movement mechanics are very easy to learn and understand. This meets my stakeholder and user requirements of having a simple and easy game. The movement also meets my design as I had algorithms for the vertical and horizontal directions of movement.

creating the attack function and animation for the player:

After creating the movement for the player I decided the next course of action would be to create the attacking method for the player.



In order to begin creating the attack animation I had to take a similar approach to how I created the player movement animation. I needed to use a collection of images stored inside of an array data structure and then these images get cycled through per frame change in order to make it look like an animation to the human eye.



Name	Date modified	Type	Size
Player_Attack_L	11/07/2020 13:30	PNG File	7 KB
Player_Attack_R	11/07/2020 13:30	PNG File	7 KB
Player_Attack2_L	11/07/2020 13:39	PNG File	8 KB
Player_Attack2_R	11/07/2020 13:39	PNG File	8 KB
Player_Attack3_L	11/07/2020 13:43	PNG File	10 KB
Player_Attack3_R	11/07/2020 13:43	PNG File	10 KB
Player_Attack4_L	11/07/2020 13:45	PNG File	9 KB
Player_Attack4_R	11/07/2020 13:45	PNG File	9 KB
Player_Attack5_L	11/07/2020 15:10	PNG File	9 KB
Player_Attack5_R	11/07/2020 15:09	PNG File	9 KB

inside the folder for attack animations i have these images which are either for the leftwards attack animation or for the rightwards animation.

```
# Attack animation for the RIGHT
attack_ani_R = [pygame.image.load("player/Attack_Animations/Player_Attack_R.png"), pygame.image.load("player/Attack_Animations/Play
                pygame.image.load("player/Attack_Animations/Player_Attack2_R.png"), pygame.image.load("player/Attack_Animations/Pla
                pygame.image.load("player/Attack_Animations/Player_Attack3_R.png"), pygame.image.load("player/Attack_Animations/Pla
                pygame.image.load("player/Attack_Animations/Player_Attack4_R.png"), pygame.image.load("player/Attack_Animations/Pla
                pygame.image.load("player/Attack_Animations/Player_Attack5_R.png"), pygame.image.load("player/Attack_Animations/Pla
                pygame.image.load("player/Attack_Animations/Player_Attack_R.png")]

# Attack animation for the LEFT
attack_ani_L = [pygame.image.load("player/Attack_Animations/Player_Attack_L.png"), pygame.image.load("player/Attack_Animations/Play
                pygame.image.load("player/Attack_Animations/Player_Attack2_L.png"), pygame.image.load("player/Attack_Animations/Pla
                pygame.image.load("player/Attack_Animations/Player_Attack3_L.png"), pygame.image.load("player/Attack_Animations/Pla
                pygame.image.load("player/Attack_Animations/Player_Attack4_L.png"), pygame.image.load("player/Attack_Animations/Pla
                pygame.image.load("player/Attack_Animations/Player_Attack5_L.png"), pygame.image.load("player/Attack_Animations/Pla
                pygame.image.load("player/Attack_Animations/Player_Attack_L.png")]
```

After creating the folder for my images I then had to make sure that my code could access these images. So what I did was that I made two separate variables, one for the left attacking animation(attack_ani_R) and one for the right attacking animation(attack_ani_R). Each variable contains the name of the corresponding image and also the directories so that my code knows where to get the image from. i also put the images inside the variable in a specific order so that the attack animation is fluid and visually correct.

```
def attack(self):
    # If attack frame has reached end of sequence, return to base frame
    if self.attack_frame > 10:
        self.attack_frame = 0
        self.attacking = False

    # Check direction for correct animation to display
    if self.direction == "RIGHT":
        self.image = attack_ani_R[self.attack_frame]
    elif self.direction == "LEFT":

        self.image = attack_ani_L[self.attack_frame]
 # Update the current attack frame
    self.attack_frame += 1
```

After sorting out the image side of things I decided to next start to create the actual attack method on the player class. At first I created an if statement for the number of frames. i set it so that if the frame goes beyond 10 the value of self.attack_frame goes back to 0. This is due to the fact that I don't want the game to crash as I have 10 images for the application and if the code tried to process anything greater than 10 the code would look for something that doesn't exist and so it would crash. i then set self.attacking to false after the 10 frames have been shown so that the attack does not repeat itself without the user inputting for the attack. After that the direction the player is moving in gets checked. if the player is moving right then the attack_ani_R variable/array is used to create the animation. This is checked through an if statement and if the player is not moving right then the code runs through the code into the elif statement whereby the attack_ani_L variable/array is used to showcast the attacking animation for the leftwards motion. after this the self.attack_frame gets incremented by one. the self.image takes the variables for the right and left animations and the animation works out due to the fact that the animation variables contain the value of the self.attack_frame variable and so whatever value the self.attack_frame variable takes will end up being the image that is displayed for that specific frame as the arrays work by having the images be indexed.

```
# Game loop
while True:
    for event in pygame.event.get():
        if event.type == QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.MOUSEBUTTONDOWN:
            if player.attacking == False:
                player.attack()
                player.attacking = True
    background.render()
    # Move player
    player.update()
    if player.attacking == True:
        player.attack()
    else:
        player.move()
    player.draw(displaysurface)
```

Before I could test the animation method for the player I had to adjust my game loop. This is because the game needs to constantly check every frame whether or not the player has decided to perform the attack just like how the game constantly checks for movement inputs. I first made some code for the detection of a left mouse click from the player to know whether or not the attacking should be done. The way this is done is through the event.type variable. if event.type == pygame.MOUSEBUTTONDOWN then this signals to the code that the player has pressed the attack button and the attacking animation should be executed. in order to avoid repeat executions of the attack function there is an if statement to see if the attack is false. if attacking is false then the player.attack() method is called and attacking is then set to true. i then added an if statement for the player attacking into the block of code under #move player so that whenever attacking is true the player.draw(displaysurface) will visually display the player attacking onto the screen.



This is the animation that is performed when the player attacks right. it's hard to capture the frames of the full animation but the animation works fine for the right side



This is the attacking animation for the player moving left and I had the same issue where it's hard to capture all the frames for the attacking animation but the animation looks good for the left hand side.

review of the creation of the attacking animation for my player:

as of now i managed to create the attacking animation for my player. I did this in a similar way to the creation of my player movement. It involved using separate images of the player performing the attack in motion and each image being displayed in their respective order based on the frame count. overall the animations do look clean and fluid enough for my

gameplay. however i do not have animations for the player moving upwards or downwards because i did not have the necessary pictures for the frames.

how i tested everything:

In order to test the code for the attacking animation I first had to try clicking multiple buttons to see if the introduction of the attacking method caused any unwanted events from occurring by me miss clicking a button or two. however nothing changed and everything remained all good for me. After that i decided to check if the actual animation worked so i was moving around and trying to attack at the same time. The animation worked and went through while the player was moving right or left.

how the section meets with the stakeholders,design and success criteria:

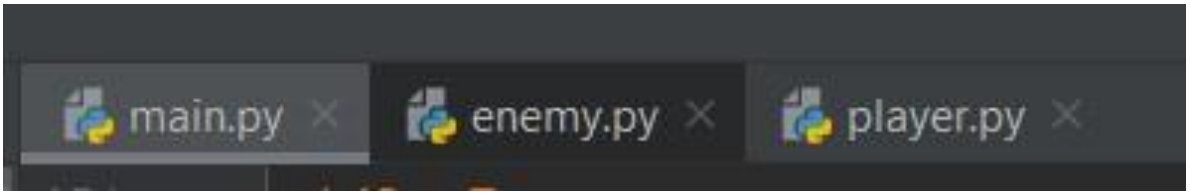
My attacking animation meets the stakeholders requirements of making the game look simple and clean. This is because the typical attacking animation is that of a sword swinging. In my design I did not include a section on attacking animations because it's hard to design the animation before the actual creation of it so there is not much I can tick off from the design section.

creating the enemies:

After creating the attack animation I realised that the next thing to make would be the enemies since the player will need something to attack. killing the enemies will be the main way the player progresses through the game and gains score

While I was creating the enemies class I started to realise that I had too much code on the same document. I had the entire player class, background class,game loop and the to be enemy class all in one document. This made it uncomfortable and long to adjust and create further code for my game. i then came to the conclusion that in order to neaten up my code and create a more friendly developing environment i should split the entire code into 3 separate scripts. one script for the player class, another for the enemy class and the final one being for the game loop and background stuff.

organising scripts before creating enemy class:



I created 3 scripts now in order to make my project a bit more organised and manageable. inside the main.py it holds the global variables and the game loop. The enemy script will contain the enemy class and the player.py script will hold the player class.

```
import pygame
import sys
from pygame.locals import *

import enemy
import player
```

I then realised that within my main.py script the game loop called a lot of my player class methods. By moving the code to separate scripts it meant that in my main.py script I needed to import the enemy and player classes into my main.py script so that the game loop has access to them. this means I can now edit the enemy class and the player class within their own scripts while still running properly.

creating the enemy class:

```
class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        # Inherit from the pygame.sprite.Sprite superclass
        super().__init__()
        self.image = pygame.image.load("2_enemies_idle_001.png")  # Find image for enemy
        self.rect = self.image.get_rect()
        xPos = random.randint(0, 1)
        if xPos == 0:
            xPos = 0
        else:
            xPos = SCREEN_WIDTH
        yPos = random.randint(100, SCREEN_HEIGHT - 100)

        self.rect.center = (xPos, yPos)
```

This is the start of my enemy class. When creating the enemy class I took a similar approach to how I created the enemy class as the enemy acts like a similar component of the game. I started out by initialising the class itself using the first two lines of the code. The class takes in the superclass pygame.sprite.sprite just like the player class and so it means alot of the already built pygame sprite methods can be accessed by the enemy class.i then created a variable for the actual image of the enemies. The variable self.image is set equal to the pygame function pygame.load with the name of the image. since the image is in the same directory as the code it means the code can access the image without me having to specify where it is in.



This is the image for the enemy that the code gets access to.

Just like the player, I then create a rectangle for the enemies through the self.rect variable. The self.rect variable is equal to self.image.get_rect() which makes it so that the enemy image has a rectangle that isn't visible but to the code it acts sort of like a boundary for the enemy and it will allow me to implement collision later on in the game.

I then created variables for the x_pos of the enemy and the y_pos for the enemy. the x_pos is equal to random.randint(0.1). this will make it so that the enemies x coordinate will be between 0 and 1. This is so that the enemy will spawn in the far side of the map giving the player time to move around and not have enemies spawning on top of themselves. The enemies spawning far away also gives the player time to make decisions with how they're gonna play.

for the y value of the enemy it uses a similar concept. The variable Ypos is equal to random.randint(100, SCREEN_HEIGHT -1). This means that the y position of the enemy will be any spot 100 from the max height of the screen . This is done so that the enemy spawn is not too predictable for the player.

then have a variable called selt.rect.center which takes in the parameters xpos and ypos. this makes it so that the invisible rectangle on the enemies is on the x and y coordinates of the enemies

```
# Combat
self.attacking = False
self.attack_frame = 0

# Set the default variables
self.health = 5
self.direction = "RIGHT"

# Set up animation variables
# self.running = False
self.move_frame = 0
self.xpos = (0, 0)
self.ypos = (0, 0)
```

These are the rest of the variables within the enemy class. These variables as of now do not have much use but they may prove of use later when I implement the enemies methods.

The next bit of developing my game was to give the enemies movement.

```
def move(self, playerCoords):
    playerX = playerCoords[0]
    playerY = playerCoords[1]
    enemyX = self.rect.center[0]
    enemyY = self.rect.center[1]
```

The movement method for the enemies takes in the parameter playerCoords. The playerCoords are the in-game players x and y coordinates. i then create a variable for player x and player y which just represents the players x and y coordinates. it works by being equal to the necessary index within the array of playerCoords. enemyX and enemyY are variables for the enemies x and y coordinates which is equal to the necessary index of the array within the self.rect.center.

```
if self.rect.left > 0:
    if enemyX > playerX:
        # self.image = run_ani_L[self.move_frame]
        self.rect.move_ip(-1, 0)
        self.direction = "LEFT"
if self.rect.right < SCREEN_WIDTH:
    if enemyX < playerX:
        # self.image = run_ani_R[self.move_frame]
        self.rect.move_ip(1, 0)
        self.direction = "RIGHT"
if self.rect.top > 40:
    if enemyY > playerY:
        self.rect.move_ip(0, -1)
        # self.direction = "UP"
if self.rect.bottom < 485:
    if enemyY < playerY:
        self.rect.move_ip(0, 1)
        # self.direction = "DOWN"
```

This is the code for the enemy actually being able to move within the game. I use a number of if statements to determine which direction the enemy will move in. I first check to see if self.rect.left is > 0. This checks to see if the enemies position is greater than 0 in terms of the map's coordinates, meaning the player is on the left side of the map. it then checks to see if the enemies x coordinate is greater than the players x coordinate. If this is the case then that means that the enemy is to the right of the player. This means that the enemy needs to

move left in order to move closer to the player since the main objective of the enemies is to kill the player. if the if statement is met then the code uses the function self.rect.move_ip(-1,0). This function takes away one from the x coordinate of the enemy which means moving the enemy more to the left.

I then have another if statement which checks if the enemies x coordinate is less than SCREEN_WIDTH which means that it checks to see if the enemy is within the screen's dimensions from the right side. if this is met then the code checks to see if the enemiesX is less than the players x coordinate. if this is the case then the enemy moves to the right in order to be closer to the player. Just like before the self.rect.move_ip() function is used to move the enemy but this time 1 is instead added to the enemies x coordinate in order to move it more rightwards.

i then made an if statement to check if self.rect.top is > 40. This is used to make sure that the enemies cannot go anywhere less than the y coordinate of 40. This is due to the fact that this is the map's restrictions. it then checks for if the enemies y coordinate is greater than the players y coordinate. if this is the case then the self.rect.move_ip() takes away 1 from the enemies y coordinate meaning the enemies moves downwards from the users point of view.

the final if statement then checks to see if the self.rect_bottom is less than 485. This is so that the enemy cannot move below y coordinate of 485 and this means the enemies cannot move too low so that they are out of the visible screen for the user. if this statement is met the enemies y coordinate is checked to see it is less than the players y coordinate. if this is the case then the self.rect.move_ip() method adds 1 to the enemies y coordinate. This will mean the enemies move upwards from the users point of view.

In general the whole point of these if statements is so that the enemies movements can be determined by the position of the player.

```
enemyGroup = pygame.sprite.Group()
```

I then had to actually create the enemies. instead of just creating an instance/object of the enemy i created an enemyGroup. This is because I will be having multiple enemies on the screen at the same time every now and then. the use of a group will make it easier for me to actually maintain this.

```
# Move enemies
for enemySprite in enemyGroup:
    # enemyGroup is a spritegroup, need to validate enemySprite as type enemy
    if isinstance(enemySprite, enemy.Enemy):
        enemySprite.update()
        enemySprite.move(player.rect.center)
        enemySprite.draw(displaysurface)
```

The enemy movement is something that will be constantly changing and updating and so there needs to be some code of the enemy movement inside of the game loop. i first created and if statement that for enemysprite in enemyGroup. this is because i will have many enemies at once and so they all will need to be checked for any necessary changes in movement.

The actual code works by checking each instance of the enemy sprite within my enemy class that has been imported into my main.py file. it performs the enemySprite.update() and also performs the enemySprite.move(player.rect.center) which causes the enemies to move towards the players centre coordinates. After all these movements the user needs to be able to see and visualise these moments and so the enemySprite.draw(displaysurface) method is used to draw the enemies movements onto the screen every frame. This will make the enemy movements viewable to the player.

spawning the enemies:

After creating the movement for the enemies I needed to create what the enemies would spawn onto the map.

```
ENEMY_SPAWN = pygame.USEREVENT + 2
pygame.time.set_timer(ENEMY_SPAWN, 5000)
```

In order to create the enemy spawning mechanism I first had to create two global variables. The ENEMY_SPAWN variable is equal to pygame.USEREVENT + 2. This is because pygame allows the developer to create their own events that can be checked for within a game loop per frame. I have used this feature in order to allow my game loop to constantly check for whether it will be necessary for any enemies to be spawned in. The pygame.time.set_timer() method is used by me so that I can set an interval for when the enemies should be spawned. Inside the parameters I set the time till the next spawn of the enemy to 5000 milliseconds. This means that from the user's perspective a new enemy will spawn after 5 real life seconds of gameplay.

```
if event.type == ENEMY_SPAWN:
    enemyGroup.add(enemy.Enemy())
```

i then added this if statement within the main body of the game loop. This statement checks for the event ENEMY_SPAWN and then performs enemyGroup.add(enemy.Enemy()). This means that in the enemyGroup an instant of my enemy class is added.

results of the enemy class and movement:



This is a screenshot of the enemies movement coming towards the player from the left side. This happened when I first started the game. the enemy moves towards the player depending on where the player is.



This is another image for the enemy movement. This image shows the enemy moving towards the player from the right hand side of the map.



This is an image for enemies moving towards the player while the player is moving upwards.

creating the player and enemy collision detection:

In order for the enemies to actually damage the player and for the player to damage the enemies i will need to implement some form of collision detection.

```
    def enemy_hit(self,player):
        if pygame.sprite.collide_rect(player,self):
            print("enemy hit")

            self.kill()
```

to allow for collision between the enemy and the player i had to add this code into my enemy class. The enemy_hit method checks for me whether or not the player makes any contact with the enemy. This is done through the existing pygame function pygame.sprite.collide_rect(player,self) which takes in the player as a parameter and it is capable of determining whether or not the player and enemy have collided. if the player collides with the enemy the self.kill() method is used. What this does is that the instance of the enemy which has made contact with the player's attack is killed and removed from the enemyspirte group.

```
hit_cooldown = pygame.USEREVENT + 1
```

In order to create the player getting hit method I first had to create a pygame userevent called hit_cooldown. This will be an event that the game loop will be able to check for every frame that goes by. The hit_cooldown event is needed in my code because when i was creating the enemy and player attacking methods in game i realised that the enemies could hit the enemy multiple times within the same second. This meant that the user would stand no chance against the enemies as a single hit from the enemies could wipe out the player instantly from the users point of view. This meant that when I was creating my player_hit method I had to be mindful of this.

```
def player_hit(self,hit_cooldown,Health):
    if self.cooldown == False:
        self.cooldown = True  # Enable the cooldown
        pygame.time.set_timer(hit_cooldown, 1000)  # Resets cooldown in 1 second

        print("hit")
        self.health -= 1
```

This is the method for the player getting hit by the enemy. It is located inside of the player class. The method takes in the parameters hit_cooldown and the health variable. If the player gets hit then the self.cooldown goes from False to True in order to prevent getting hit by the enemies any more times than intended within the same second. The pygame.time.set_timer(hit_cooldown, 1000) method is used to determine how long the cooldown is active for. the code then after registering the player getting hit will decrement the players health by 1.

```
if event.type == hit_cooldown:
    player.cooldown = False
    pygame.time.set_timer(hit_cooldown, 0)
if event.type == ENEMY_SPAWN:
    enemyGroup.add(enemy.Enemy())
```

i then added my hit_cooldown to the game loop which allows for the event hit_colldown to be checked for within every frame. if hit_cooldown is detected it sets player.cooldown to false and uses the pygame timer.

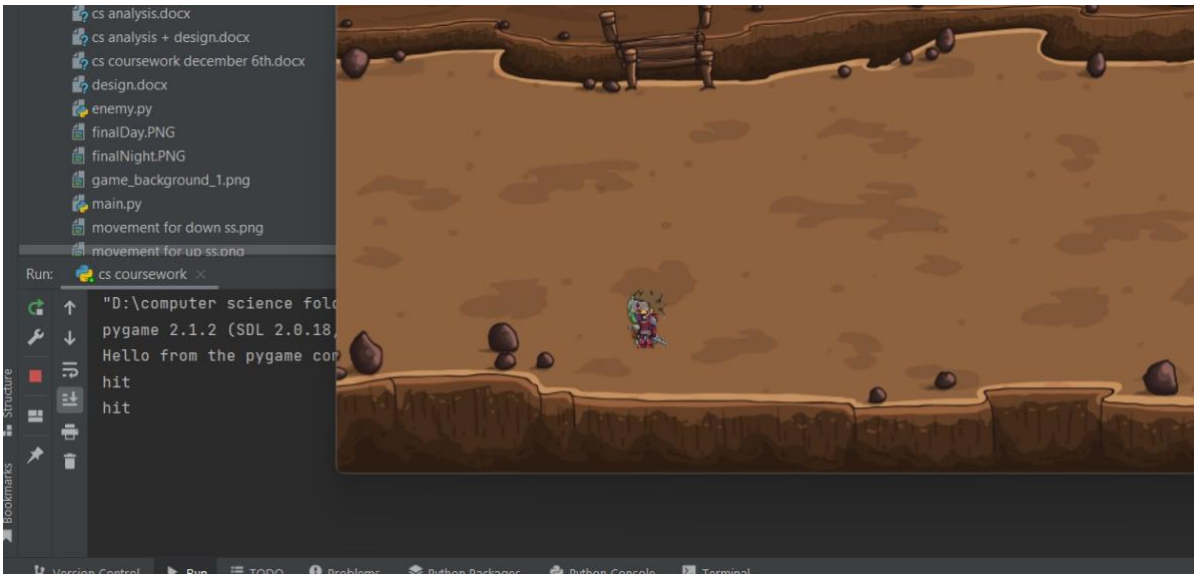


This is a screenshot of what happens when the player gets hit by the enemies. As of now it's hard to tell what is going on due to the lack of a health bar but the player is taking damage for every second the player is on the enemy.

review of creating the enemies and enemy related functions:

In this section of the development process I have created the enemies and all the enemy related functions. i first managed to create the enemy class and then implement an enemy movement system whereby the enemy moves towards the players directions. I then created the enemy attacking system where the enemy attacks the player and subtracts one from the health. I then also managed to create an enemy sprite group system where many enemies can be alive at the same time and get killed by the player. **how i tested everything:**

While creating the entire enemy related section of my code the main form of testing was for me to constantly alter the code and see the outcomes of the code by running the program. for the enemies movements i ran the code and at the same time i purposely moved in different directions with the player to see how the enemies respond to the change in movements. in order for me to test if the enemy attack worked i used the print function to print whether or not the player was hit



on the screenshot it can be seen that the enemy is on the player. This means that the player is getting hit by the enemy by the game's logic. In order for me to actually know I can see that the player is getting hit through the print statements which print to the console “hit” which notifies me that the player is getting hit.

In order to know if the player gets killed and damaged by the enemies I ran the code and just allowed the enemy to be on top of the player for 5 seconds meaning the player should take 5 damage. This will mean that the player should die since the player has only got 5 hitpoints. In my test I saw the game end after 5 seconds meaning that the player has been killed. **how the section meets with the stakeholders,design and success criteria:**

while creating the whole enemy funtionality i took into account the stakeholder, design and the success criteria points. The look of the enemy meets the design section as I used the exact image/design of the enemy in my enemy creation. The enemies matched the whole theme of how I specified the enemy will look because the enemies are clearly distinguishable between the players. The enemy also holds a weapon which I specified would be the case when I created the design for the game. creating the enemy attacking function and the player attacking the enemies meets the overall theme of the game specified in my design and stakeholder meetings. This is because I specified earlier that the main objective of the game is for the player to attack the enemies and kill them.

creating the score aspect of the game:

After creating the enemy and the player attacking methods I then had to work on creating the actual score for the game. The main idea behind the score is that after killing an enemy the score will go up and be displayed on the screen.

```
score_value = 0
textX = 800
textY = 10

def show_score(x, y):
    score = font.render("score :" + str(score_value), True, WHITE)
    displaysurface.blit(score,(x,y))
```

When creating the score method for my game I first had to create some global variables. The first one is score_value. This variable is used to initialise the score of the player by setting it to 0. I then created global variables called textX and textY which will be used to determine the x and y coordinates of the actual position of the score on the users screen. I set textX to 800 and textY to 10. This is because I want the score to be displayed on the top right hand side of the screen. The show_score method takes in two parameters x and y which when I call it later on will take the place of textY and textX. inside the function i have score = font.render(“score :” + str(score_value) , True, WHITE) this line of code uses the render function i created in my code earlier on when creating the background class and renders the contents of the code to the screen. I have the “score :'' in string format which will mean that the player will be seeing this on the screen and next to this will be str(score_value) which will display in integer form the current score of the player. I then have the line display surface.blit(score,(x,y)) which takes in the x and y parameters and blits the score to the screen of the player.

```
for enemySprite in enemyGroup:
    enemySprite.enemy_hit(player)
    score_value += 1
```

I then had to make sure there was some way the code new to add 1 to the score each time an enemy was killed. This is why I added this block of code into the game loop as the score going up is like an event the loop should check for every frame of the game in the event the player does kill an enemy. What the actual code does is that for every enemySprite in the enemy group, if the enemy_hit is called then that means the enemy has been beaten by the player and so in turn is considered dead. This means that the score should increase and so the score_value variable increments by 1.
At the bottom of the game loop there is a line of code where it says pygame.display.update() which allows for the display to constantly be updated and display what the code is trying to show.



This is the result of the score method being used. at the start of the game there is a score bar which will display the score of the player depending on how many enemies they kill

creating a health bar:

After creating the score I realised that the next thing was to create the health bar.

```
health_ani = [pygame.image.load("health/heart0.png"), pygame.image.load("health/heart.png"),
              pygame.image.load("health/heart2.png"), pygame.image.load("health/heart3.png"),
              pygame.image.load("health/heart4.png"), pygame.image.load("health/heart5.png")]
```

In order to create the health bar for my game I needed to create a health_ani variable and put an array within the variable containing the necessary images of the health bar. This is because the health bar will work like an animation whereby as the player takes damage the health bar will decrease for the player's screen and this will be done by the removal of a heart from 5 in total. In the array I put the images in order of how many hearts there are so that the value of self.health will represent the required image of the hearts.

```
class HealthBar(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.image.load("health/heart5.png")

    def render(self):
        displaysurface.blit(self.image, (10, 10))
```

In order to create the health bar I created a class for the health bar. I decided to include this class within the player.py because it made the most sense to do so since the health bar corresponds to the player mainly.

First I had to initialise the class and then I decided to create a variable within the class called self.image. This variable will be used to load the image of the health bar at 5 hearts using the pygame.image.load function. within this function i had a directory to the heart image that's in my folder.

I then created a render method within the health class so that it can be used to render the health bar to the coordinates (10,10) which for the player is the top left of the screen.

```
Health = HealthBar()
```

after creating the health bar class i then had to create an actual object of the class using this line of code above. this will initialise the health bar and make it usable by the code.

```
def player_hit(self,hit_cooldown,Health):
    if self.cooldown == False:
        self.cooldown = True  # Enable the cooldown
        pygame.time.set_timer(hit_cooldown, 2000)  # Resets cooldown in 1 second

        print("hit")
        self.health -= 1
        Health.image = health_ani[self.health]
```

After initialising the health bar I realised that the health bar will need to be changed as the player takes damage in order to show the amount of health the player has remaining. This is why in the palyer_hit method I added an extra line after selfhealth -= 1. The variable is called health.image and it is equal to my array of health images called health_ani. inside health_ani I put in self.health which means that the value of self.health will be the index of the array of

images for my health bar. This will mean that when self.health is 5 the 5 hearts image will be displayed and when self.health is 4 the 4 hearts image will be displayed etc.

```
    if player.health > 0:
        displaysurface.blit(player.image, player.rect)
Health.render()
```

I then added this block of code into the game loop. This code is used to check if the player's health is greater than 0. if it is then the player will be displayed to the screen. the health bar is then rendered onto the screen.



This is the result of the creation of the health bar. The bar is on the top left of the screen like intended and looks just fine for the nature of my game.

review of creating the health bar:

In this section I managed to create the health bar for my game. I did it through the use of a health bar class. The class contained a variable for the images of the health bar inside of an array. I then also had to create the code for the game recognising the decrease in health and displaying the necessary images. **how i tested everything:**

In order to test the health bar my first aim was to create the code and then see if the code works and a health bar loads up in the first place. This bit was successful and a health bar did load up. My next way to test the health bar functionality was to allow the enemies to hit me and then see whether or not the health bar gets adjusted.



After allowing the enemies to hit me it was evident that the health bar works as the health was decreasing. to do some more further testing i then had to check whether or not the game would end after all the hearts were gone. This came out to be successful as once all the hearts disappeared my game window ended. **how the section meets with the stakeholders,design and success criteria:**

The creation of a health bar matches the overall design of my game. This is because the health bar is a key aspect of the game since it shows how much the player can get hit till the game ends. I also used the exact images for my health bar that I showed in the design. The health bar meets my stakeholder requirements as well because I created a health bar that will be easily distinguishable and understandable for my users.

creation of the intro and game over screen with clickable buttons:

The main body of my program has been completed. All that's left is a few additional things to make the user's experience that much better.

I decided that I should create an intro screen for my game which gives the player the options to play the game or quit the game.

In order to create the intro and game over screens of my game I have to first make clickable buttons which the user can press depending on what they want to do.

```
class button:
    def __init__(self, x, y, width, height, fg, bg, content, fontsize):
        self.font = pygame.font.Font("square.ttf", fontsize)
        self.content = content

        self.x = x
        self.y = y
        self.width = width
        self.height = height
        self.fg = fg
        self.bg = bg

        self.image = pygame.Surface((self.width, self.height))
        self.image.fill(self.bg)
        self.rect = self.image.get_rect()

        self.rect.x = self.x
        self.rect.y = self.y
        self.text = self.font.render(self.content, True, self.fg)
        self.text_rect = self.text.get_rect(center=(self.width / 2, self.height / 2))
        self.image.blit(self.text, self.text_rect)
```

In order to create the clickable buttons for my intro and game over screen I had to create a button class. I first initialised the class itself and I put in parameters for x,y,width,height, fg, bg, content and font size. all these will come into effect when i want to create multiple buttons/instances of the class. I then start to create the variables inside of the class. self.x is equal to the passed in value of x and the same goes for self.y. the self.y and self.x variables are used by the class to determine the x and y coordinates of the button to be made. I then have self.height and self.width variables which will be used to determine the width and height of the buttons i make. The variables self.fg and self.bg are used to set the colours of the font on the button and the colour of the background of the button. I then have a variable called self.image which is equal to pygame.surface(self.width, self.height). This line of code is used to display the button to the screen using the desired width and height of the programmer. the variable self.image.fill() will be used to fill the button with whichever colour the developer specifies inside of self.bg. I then have a variable called self.rect which is used to create a rectangle around the variable self.image() which is the button in reality. i then create the rectangle for the necessary dimensions by using self.rect.x and self.rect.y. the self.text() variable is used in order to render the desired content of the button onto the screen with the desired font and colour. i then use self.text_rect in order to create a rectangle for the button. this will help allow for me to actually click the button later on. I then used self.image.blit to blit the self.text and the self.text_rect to the screen.

```
def is_pressed(self, pos, pressed):
    if self.rect.collidepoint(pos):
        if pressed[0]:
            return True
        return False
    return False
```

After creating the variables for the button class I created this method which uses the pre-built pygame function .collidepoint() to check the position of the mouse cursor from the pos parameter and then use this to work out whether the mouse has been clicked or not.

After creating the button class and the is_pressed method I could now create the intro and game over screens.

```
class game:
    def __init__(self):
        self.running = True
        self.font = pygame.font.Font("square.ttf", 32)
        self.screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
        self.intro_background = pygame.image.load("finalDay.png")
```

In order to be able to create an intro screen the first thing i decided to do was to create a class called game which will hold the intro screen and the game over screen methods inside of it. I first initialised the class and then I created the attributes of the class. I first created a variable called self.font. This variable is equal to the pygame.font.font() function which will allow me to use any chosen font in this class. I have decided to use a font that looks sort of box-like in order to resemble the writing of a game. the font will be of size 32 as shown within the parameters of the variable. I then have a variable called screen which will just be used so that I can display the intro and game over screen using my global variable screen width and heights. I then have a variable called intro_background. this variable will help me to set the background image onto the screen.



This is the image I will be using for the background of the game over an intro screen.

i got this image off the web for free of the website
https://blank-canvas.itch.io/parallax-pixel-art-background-desert

This website allows the users to use this image for free in projects. The reason I chose an image like this for the background of my game is because I think it suits the genre of the game. This is because the inside background of my game looks like the player in some desert-like place in the middle of nowhere. This is why for the intro screen I thought it would be suitable to use a picture of the desert to help match the whole design of the game's background allowing the user to feel like the intro screen is a preview of the scenery of the actual game.

```
from player import button, Player, HealthBar
```

Before I could use the buttons in my code I had to first import the button class which was held in my player.py into my main.py document as I did not have all the classes in the same document and so they were not all directly accessible.

```
def intro_screen(self):
    # display variable controls the menu game loop
    display = True
    title = self.font.render("PYTHON GAME", True, WHITE)
    title_rect = title.get_rect(x=10, y=10)
    play_button = button(10, 50, 100, 50, WHITE, BLUE, "play", 32)
    quit_button = button(10, 110, 100, 50, WHITE, BLUE, "quit", 32)
```

I then started to create the method for the intro screen of the game. In this method I created a title variable which uses the font.render() function in order to render whatever I set it to onto the screen. This method also allows me to use the font I specified inside of the class attributes when I create the title of my game. inside of font.render() i can set the name of the game as whatever i want it to be with whatever colour i want it to be which in this case is WHITE. I then have a variable for title_rect which creates a rectangle around the title of my game. After that I decided to create two different buttons. The first button was play_button. in order to create the button I can easily call the button class and pass in the necessary arguments and this will create the button in my desired way. I did the same thing for the quit_button by calling the button class and passing the necessary arguments into the class to create my button. I called the buttons play and quit with both of them having the same text colour of white and background colour of blue.

```
while display:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            display = False
            self.running = False

    mouse_pos = pygame.mouse.get_pos()
    mouse_pressed = pygame.mouse.get_pressed()
    if play_button.is_pressed(mouse_pos, mouse_pressed):
        # Close the menu object and start the game
        display = False
    if quit_button.is_pressed(mouse_pos, mouse_pressed):
        pygame.quit()
        sys.exit()

    self.screen.blit(self.intro_background, (0, 0))
    self.screen.blit(title, title_rect)
    self.screen.blit(play_button.image, play_button.rect)
    self.screen.blit(quit_button.image, quit_button.rect)
    pygame.display.update()
```

This is the rest of the code for the intro_screen. The code checks for the pygame event pygame.quit which if true the display changes to false and self.running changes to false.

I then created some variables within the while loop. The variable mouse_pos is equal to pygame.mouse.get_pos() which is an existing pygame function that can get the position of my mouse's cursor and use it within the code. i then have a variable called mouse_pressed that checks if the mouse button is pressed as the while loop is running.

i then have an if statement that checks if the play_button is pressed and it takes in the parameters mouse_pos and mouse_pressed. the if statement is met and the play_button is pressed then the intro screen display is set to false and so the game screen will be displayed. On the other hand if the quit_button is pressed then the lines pygame.quit() and sys.exit() will be used which causes the users game window to close therefore ending the game.

Within this block of code I then had to blit a bunch of things onto the screen such as the intro background,title ,play_button and the quit_button with their respective rectangles. This is so that the user can actually see the buttons and intro background on their screens. I then have a pygame.display.update() line in order to constantly be updating and showing the most recent changes to the window.

```
def game_over_screen(self, player, Health):
    global score_value
    # display variable controls the menu game loop
    display = True
    title = self.font.render("GAME OVER", True, WHITE)
    title_rect = title.get_rect(x=10, y=10)
    play_button = button(10, 50, 100, 50, WHITE, BLUE, "play", 32)
    quit_button = button(10, 110, 100, 50, WHITE, BLUE, "quit", 32)
    score = self.font.render("Score:" + str(score_value), True, WHITE)
    score_rect = score.get_rect(x=200, y=60)
```

I then decided it was time to move onto creating the game over screen for my game. In order to do this it took a similar approach to how I created the intro_screen. The difference in the game over screen is that aside from the play and quit buttons I also had to add in the whole score aspect as the final score needs to be displayed onto the screen. in order to create the final score i had to add in the global variable score_value. This will be used later on to display the score. I then set the display to True. After that I created a variable called title which was equal to self.font.render(“GAME OVER”,true,white). This line of code will be what text gets displayed onto the screen and so in this case game over is what the user will see on the screen. I then have a variable called title_rect which creates a rectangle around the title of my game so that I can introduce the clickable aspect to it. After this I created the play_button using the button class I imported earlier on. The play_button for the game over screen is the same as the one in the intro screen and so I will be using the same parameters. I then have a button for the quit game which also is using the same parameters as the one in the intro screen. I then have a live code called score which is used in order to render the

score onto the screen with the desired font using self.font.render() adn passing in the necessary parameters. I then have a line of code called score_rect which creates the rectangle of the score at the desired x and y coordinates.

```
while display:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            display = False
            self.running = False

    mouse_pos = pygame.mouse.get_pos()
    mouse_pressed = pygame.mouse.get_pressed()
    if play_button.is_pressed(mouse_pos, mouse_pressed):
        # Reset all key game variables
        score_value = 0
        player.reset_health(Health)
        display = False
    if quit_button.is_pressed(mouse_pos, mouse_pressed):
        pygame.quit()
        sys.exit()
```

This is the next block of code within the game over screen. The code checks for the event pygame.QUIT which is the player pressing the close window button on the top right of the screen. if the player does this then the display becomes false and self.running becomes false. I then have some variables within the loop. The first variable is mouse_pos. This variable is equal to pygame.mosue.get_pos() which is an existing pygame function that gets the cursor position of my mouse. The next variable was mouse_pressed which is equal to the pygame function pygame.mouse.get_pressed(). This function allows the code to know when the mouse button is clicked by the user. I then created an if statement which checks for if the play_button is pressed and if done then the code initialises the key game variables. This consists of the score_value being set to 0 and the player.reset_health method from the player class is used to set the health back to 5. I then set the display to false so that the game over screen would disappear and allow the player to play the game.

The next if statement checks for if the player has clicked the quit_button using the is_pressed method which takes in the mouse pos and mosue_pressed variables as parameters. if the statement is met and the quit button is actually pressed then the game window will close and end the game.

```
self.screen.blit(self.intro_background, (0, 0))
self.screen.blit(title, title_rect)
self.screen.blit(play_button.image, play_button.rect)
self.screen.blit(quit_button.image, quit_button.rect)
self.screen.blit(score, score_rect)
pygame.display.update()
```

After finishing the while loop i had just blit everything onto the screen so that the user can actually see the buttons and the background onto the screen just like the intro screen. the only difference is that on the game over screen everything such as the title, play button and quit button are blit onto the screen but also the score is blit onto the screen.

```
gameMenu = game()
gameMenu.intro_screen()
```

In order to actually utilise the code within my game class I had to create the object of the class somewhere outside the class. The gamemenu.intro_screen() line of code loads up only the into screen and allows the player to decide if they want to play or quit the game.

```
# Player and enemy collision behaviour
if pygame.sprite.spritecollideany(player, enemyGroup):
    if player.attacking == True:
        print("attack")

        for enemySprite in enemyGroup:
            enemySprite.enemy_hit(player)
            score_value += 1
    else:
        player.player_hit(hit_cooldown,Health)
        if player.health == 0:
            gameMenu.game_over_screen(player, Health)
    if player.health > 0:
        displaysurface.blit(player.image, player.rect)
Health.render()
```

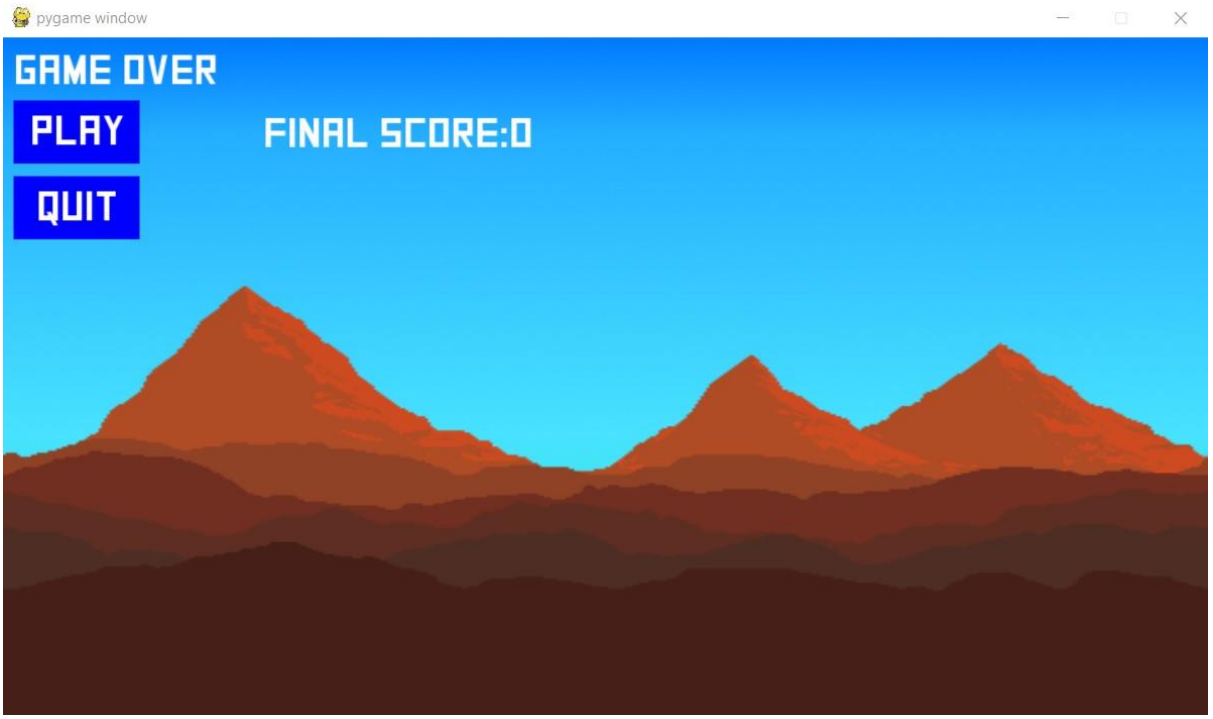
In order for the game to display over screen I had to put some extra code into the game loop. The code goes within the player and enemy collision section. The thing i added is that if the collision between the player and the enemy is not an attack it means the player has taken damage and so therefore the player has for hit and lost health. the code then checks if the health is 0. if the health indeed is 0 then the game over screen method from the gameMenu object is displayed into the screen.

intro screen:



this what the user will see of the intro screen as of now when the code is run

game over screen:



This is what the user will see as the game over screen once they die within the actual game. The game over screen is fairly similar to the intro screen but also displays the score near the play and quit buttons.

review of creating the game over and intro screens:

In this stage of development I have created the game over and the intro screen of my game. in order to do this i first had to create clickable working buttons the user can press which go on the intro and game over screens. In order to create the clickable buttons I used a class and then created objects of the class. I then had to import the button class into the document which contained my game screen class. I then used the two to create the game over and intro screens which the user can see. **how i tested everything:**

In order to test the intro and game over screens I first had to make sure the background loaded up on the screen. This was successful and then I had to create the actual buttons. While creating the buttons I tested the code by running it and seeing if any errors came up. I then created the buttons and displayed them on the screen. Before creating any more code during this stage I had to test to see if the buttons load up on top of the display and are visible by the player. The reason I did this is because some errors warise when displaying things, for instance the background may be displayed on top of the buttons making the buttons invisible to the player.

In order to test the buttons I tried to click them and see what would happen when I did so. I also tried to put in wrong inputs such as the right click or using any keys in order to see how the program would respond. the results were that the button only takes in the intended button which is the mouse button click. I then allowed myself to die within the game just to see if the game over screen did pop up which it did. i then tested if the buttons on the game over screen worked by clicking them and seeing what they do **how the section meets with the stakeholders,design and success criteria:**

This section meets my stakeholder requirement of having a simple and easy to understand interface. This is because it's very easy to understand what to do when looking at the intro screen. The buttons are also effective and do indeed do their intended purpose. The design of the game over screen is fairly similar to my current solution as the screens continue a play and quit button but the general position of the buttons is different to my design as within my solution the buttons are on the left hand side of the screen but in my design they are in the centre of the screen.

review of the code for potential runtime errors:

```
# Move enemies
for enemySprite in enemyGroup:
    # enemyGroup is a spritegroup, need to validate enemy

        enemySprite.update()
        enemySprite.move(player.rect.center)
        enemySprite.draw(displaysurface)

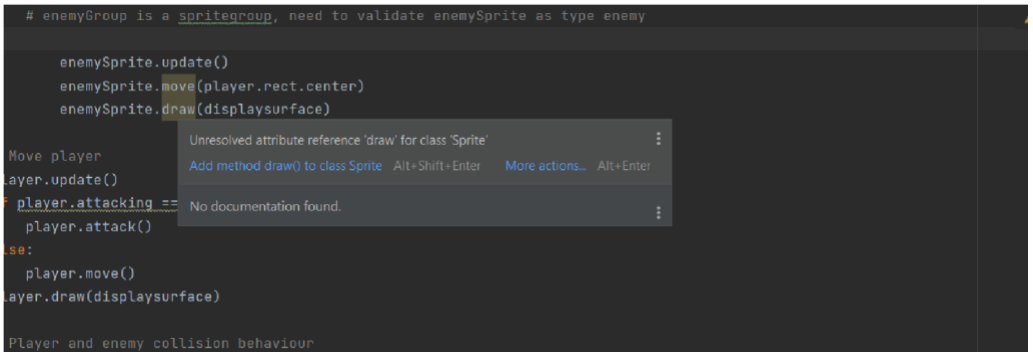
# Move player
player.update()
if player.attacking == True:
    player.attack()
else:
    player.move()
player.draw(displaysurface)

# Player and enemy collision behaviour
if pygame.sprite.spritecollideany(player, enemyGroup):
    if player.attacking == True:
        print("attack")

        for enemySprite in enemyGroup:

            enemySprite.enemy_hit(player)
            score_value += 1
    else:
```

As I was reviewing the entire code as a whole I came across some errors. I'm using an IDE called pycharm in order to develop my program. The IDE has a feature where it can identify potential runtime errors and highlight them in yellow. In this case some things within my game loop were highlighted in yellow and so I looked into why the problem was occurring.



This is the reason behind the potential runtime error. The error is occuring because the code being run is intended to be applied only to the enemies within the game. however the code is being applied to the sprite class and in this case the enemies aren't the only spirited the player inside of the game is also considered a sprite

```
    if isinstance(enemySprite, enemy.Enemy):
        enemySprite.update()
        enemySprite.move(player.rect.center)
        enemySprite.draw(displaysurface)

# Move player
player.update()
if player.attacking == True:
    player.attack()
else:
    player.move()
player.draw(displaysurface)

# Player and enemy collision behaviour
if pygame.sprite.spritecollideany(player, enemyGroup):
    if player.attacking == True:
        print("attack")

        for enemySprite in enemyGroup:
            if isinstance(enemySprite, enemy.Enemy):
            enemySprite.enemy_hit(player)
            score_value += 1
```

in order to fix this error i just had to add an extra if statement above the enemy functions. This if statement checks ifinstance(enemysprite,enemy.enemy) meaning that it first checks to validate if the instance is of the in-game enemies and then continues on with the code. This ended up fixing the error as the code that could have resulted in a run time error is no longer highlighted yellow.

last stage of testing

As my development has now finished I can now conduct the post development test i created in the design section and also do any additional tests.

function being tested:	justification for it being tested:	expected outcome:	actual outcome of test:
game loads	i will need to know if the game will load up properly first of all	the game should load up correctly	works correctly
map dimensions are	i will need to check	the map should be	works correctly

correct	whether or not the map loads up correctly and fits the size of my games window correctly	the exact size of my game window	
player image and mob image have the correct dimensions	the player and the the mobs will need to have the correct proportion to the mao and each other	it should look fine to the users eyes	works correctly
mobs spawn and in correct amounts	the mobs will need to spawn in correct amounts and at places so the player does not get overwhelmed	mobs should spawn 1 every 5 seconds	works fine
mobs should be able to deal damage to player	the mobs are meant to be able to deal 20% of the players damage per hit	should work	works correctly
the health bar and score appear on the top of the screen	i will need to make sure that the health bar and the score loads up and is in their intended location on the screen	the health bar should be on the top left of the screen and the score bar should be on the top right of the screen	works correctly
the health bar adjusts when the player gets hit	i will need to make sure the health bar shows the correct amount of health the player has remaining	the health bar should reduce by one heart every time the player gets hit	works correctly
the score should correctly adjust	i will need to make sure that the score bar displays the correct score the player has	the score bar should increase by one each time and enemy is killed and it should be shown on the screen	works correctly

the player can move in the users intended way	i will need to make sure the in game player moves in the right direction the	player should end up moving in the intended way	works correctly
	user inputs		
the movement animation works	i will need to make sure that the correct animations are displayed for when the player moves	movement animation should look correct	works correctly
the player cant exceed map boundary	i will need to make sure the player stays within the boundary	the in game player shouldnt be able to move outside the visible screen and be out of the users vision	works correctly
the player attack animation works	i will need to make sure the player attack animation works properly	the in game player animation should lick visually appealing and suitable	works correctly
when the player attacks a mob it should die	i will need to make sure that when the player attacks the mob it dies and gets off the screen	mob disappears after attack	works correctly
when the players health is 0 game ends	i will need to make sure that when the players health reaches 0 them game over screen shows up	game ends when player has no health	works correctly
player can quit or start a new game on game over screen	i will need to make sure the user can play again or quit the game on the game over screen	game over screen contains two options to play the game and quit	works correctly

further testing taken from design section:

what's being tested:	justification for testing it	result of test:
the entire game should run	i will need to make sure the entire program as a whole can run properly	the entire game does run properly with no run time errors as of now
bug testing	I will be looking for bugs in	during the creation of the
	my game. this is because my game may end up running but have some issues with it	program i did find bugs and i corrected them throughout the stages of development and showed the changes
everything looks aesthetically pleasing	i will need to make sure that once everything has been created it should look good to the users	the game as of now looks aesthetically pleasing and suitable for the games requirements
can be run/opened on multiple different devices	i will need to make sure the game can be opened and run on a number of computers so that i know it will be suitable for my stakeholders	I have tried to run the program on a few different computers and so far it runs fine as long as the computer has a python interpreter and decent enough specs.

testing for evaluation:

I will need to test my overall program to make sure that all the functions are working in the intended way and to make sure the program is robust overall. **movement function testing:**



This is a picture to show what happens when I press the w key on the keyboard. The intended function is that when I press w the player should move up and so as the screenshot shows the function is clearly working as the player is above the centre. This is done all the while the game is aware about not letting the player exceed the height and width boundaries defined in the code.



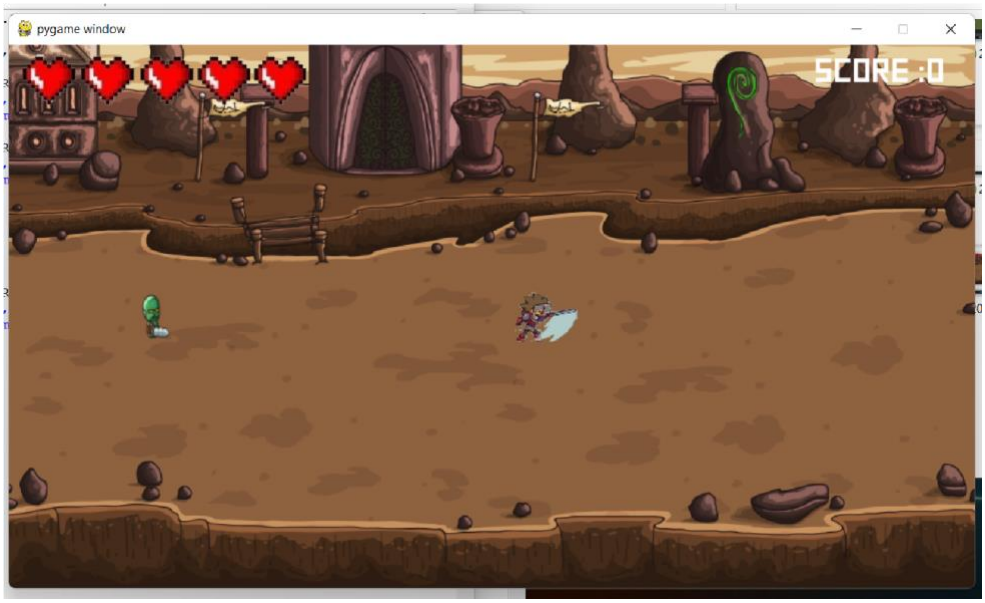
This is a screenshot for when the user presses D and the player moves right on the screen. The intended function is that the in game player moves to the right with the necessary animation using the images from the array. This clearly works as shown in the screenshot. This is done all the while the game is aware of the height and width boundaries not allowing the player to exceed them.



This screenshot shows the player moving left. The function works by causing the in-game player to move to the left direction when the user pressed the a key and also perform the right animation . The function clearly works as the screenshot shows that the player is moving to the left while in some sort of animation. This all happens while the game validates the map's borders and makes sure that the user cannot exceed the visible map.



This screenshot shows the player moving down. The intended function is that when the S key is pressed the player moves down. The screenshot shows that clearly the intended function works as the player does indeed move down. There currently is no animation for the player moving downwards as I did not include one in the code. **testing the attack function:**



This is an image showing what happens in my game when the player presses the attack button which is the left mouse button. The animation for the function clearly works as shown by the image. I also tried the attacking function while the player faces left and the animation also works then. in order to check if the attacking function as a whole worked i had to test it out on the enemies. When I conducted the test the function worked as expected because as soon as I went next to the enemy and pressed the mouse button the enemy disappeared from the screen indicating it had been killed. overall the attacking function seems to work and seems to be performing in its intended way. **the robustness of my program:**

My program overall is fairly robust. This is because it can handle most unintended inputs and deal with them correctly. for instance if the user accidentally presses keys that are not used in my code then they will not affect the gameplay. Also if the player presses the wrong mouse button while they intend to attack the attacking function will not be activated as the specific functions know to act only when the correct designated button is pressed. This means the user will not be able to move the character or attack using the character by pressing some random buttons which are not meant to do anything in the first place. the in-game character cannot also move off the visible map accidentally because the game validates whether or not the player is within the boundaries of the map so this means if the user accidentally holds onto any of the movement keys for too long the in-game character will still remain within the map. My program is also robust because of the fact that the border functions in my code prevents any users from going out of bounds. This is because in their code the movement function is based on the condition that the player's current location must be less than the maximum height and maximum width in order for the movement function to actually work. This means that the players can never move off of what is visible on the screen. **usability and stakeholder testing to inform evaluation:**

I will be asking my stakeholders a series of questions in order to find out how- my game feels to them overall and to also get feedback on the usability features of my game.

questions i will ask:

1. How easy did you find starting a game?
2. Did you find the controls of the game easy to understand?
3. Did you find aspects of the game familiar or memorable?
4. when accidentally pressing unintended buttons did the game crash or struggle to handle the input?
5. did you understand how the score system worked and how score is gained?

i asked some of my stakeholders these questions and these were the responses i got from them:

Response from barry:

question 1: How easy did you find starting a game?:

I found it quite easy to start a game as the buttons were big and it was obvious that I had to click the play button on the top left of the screen in order to start the game. This was easy for me because as soon as I pressed the button there was no further action needed and the game started.

question 2: did you find the controls of the game easy to understand?:

When I loaded into the game I found it fairly easy to understand how the game worked. I knew that traditionally people move the characters with W,A,S,D or the arrow keys so I tried both of them. I then understood how the game works by pressing those buttons and learning what each one of them does. The attacking controls were also fairly easy to understand as it was just the left mouse button and once I neared enemies I pressed the button.

question 3 : did you find aspects of the game familiar or memorable?:

the overall felt familiar to me yet i didn't ever play it. This is because the game uses traditional mechanics and it meant that the users were able to understand how to play the game. The health bar was also familiar as it used red hearts and that easily let me know that the in-game player has 5 health points. The enemies also looked like zombies and so I knew straight away that those are what I should try to kill.

question 4: when accidentally pressing unintended buttons did the game crash or struggle to handle the input?:

When I was testing out the mechanics of the game I did press buttons that are outside of the specified controls just to see what would happen. The game handled this pretty well as the program did not crash or feel sluggish due to the game struggling to handle the inputs. overall the game could handle errors pretty well. **question 5 : did you understand how the score system worked and how score is gained?:**

When playing the game I understood how the score system worked fairly well. For example, I knew that a higher score denoted that you are doing better in the game. I also knew that the score increases as the player gets more kills. overall the system is easy to understand

response from jamie:

question 1: How easy did you find starting a game?:

I found it quite easy to start a game as all i had to do was press the play button on the top right of the screen. it was fairly obvious that the play text was a clickable buttons dn the process of the game starting and loading up was easy and quick

question 2 : did you find the controls of the game easy to understand?:

At first I was confused at what buttons to press and how to actually play the game. It was only after a few attempts of playing the game till I understood the controls. This is why I feel it would be good if you introduced a quick tutorial for the user to look through to get an understanding of what to do. Other than that it was pretty easy to play the game. **question 3 : did you find aspects of the game familiar or memorable?:**

The game did indeed feel very memorable and familiar to me even though I have never played this game specifically. This is because there were things like the W,A,S,D controls which most games utilise and also the fact that the attack button was the left mouse click button. The fact that the health bar was also made of red hearts was familiar as that easily told me that the in-game player had 5 health points. The enemies also looked like traditional enemies as they were zombie-like so I knew that they were the enemies I had to fight.

question 4: when accidentally pressing unintended buttons did the game crash or struggle to handle the input?:

It was evident to me that the game could handle errors. This is because whilst I was trying the controls I sometimes clicked buttons which were not part of the controls but the game managed to handle the miss input pretty easily. I also sometimes pressed the right mouse button instead of the left when trying to kill the enemies however this did not result in the enemy dying rightly so since the specified button is the left mouse button only.

question 5 : did you understand how the score system worked and how score is gained?;

When playing the game it was fairly easy to understand how the score system worked. This is because it was clear that the score was shown on the top right of the screen. I could easily infer that the higher the score the better I am at the game. I could easily tell that the score was gained through killing the enemies.

review of the usability testing:

From what I have gathered from this questionnaire the game overall felt easy and simple to play. My users found the aspects of memorability and familiarity in order to quite quickly understand the game's features and functions. My users also found it helpful that the buttons in order to start and quit the game were very obvious to distinguish between. They liked the fact that the game loads almost instantly after hitting the play button. My users also liked the error handling aspect of the game since whenever they would press the wrong buttons the game wouldn't crash or cause any unexpected movements. my users also liked the simplicity of the score system as it was very simple in nature and worked as expected when i was creating the game

One of my users said that a quick 5 second tutorial may be necessary just for complete beginners/new players to be able to easily understand the already simple controls just to save that extra time from having to experiment. he also suggested that there be a skip buttons anyways so those who don't want to go through the tutorial can skip it. My users also said that I should maybe add some customizability to the game. for example by allowing the user to use custom skins or even have different maps to play on. This is so that the user can have a bit of variety when playing the game. They also told me to maybe implement different types of mobs so that the player can be exposed to more challenging situations rather than facing the same type of enemy each time. all of this which has been said can definitely be used by me in order to improve the game further and make it a better experience for those who play it

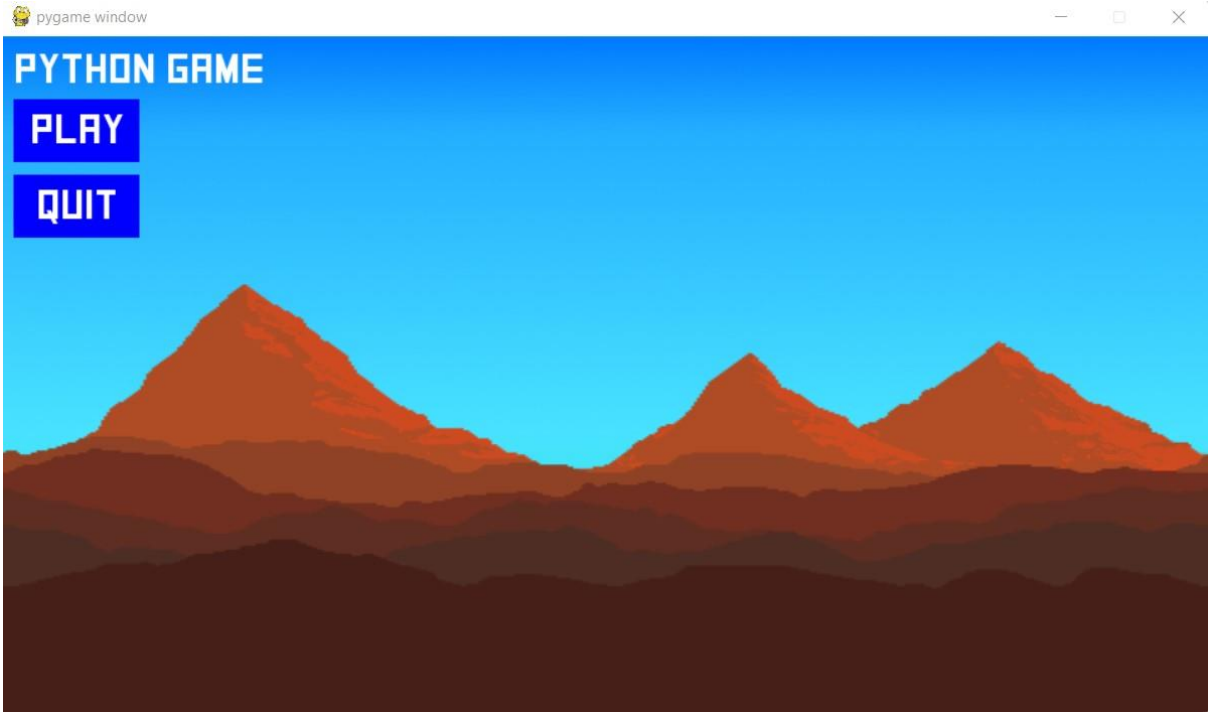
evaluation for the solution:

After finishing the solution I must evaluate the entire thing as a whole and so now I must use my success criteria from the analysis section to see if my code matches it.

success criteria:	met or not:
home page	met
suitable GUI	met
option to start a game within home page	met

option to open settings	not met
option to pause mid game	not met
option to adjust sound	not met
option to change controls	not met
simple design overall	met
the character can move within the game	met
the character has a animation while moving	met
the character can collect coins/coin count	not met
the characters health bar	met
the character taking/dealing damage	met
the map needs to be of a suitable size	met
mobs and character need suitable colour/looks	met
mobs can attack players	met
a score count	met
there should be an option to leave the game once paused	not met
the game over screen once the the player dies	met
the score system	met
option to return to homepage or start a fresh game	met partially

evidence for the success criteria:



This is the intro screen that loads up now that the game has been made. This screen loads up fine and has working buttons. The design of the intro screen is good enough for the genre of the game and the overall look of the screen is suitable for the nature of the game.

criteria points met within the screenshot:

- suitable GUI
- home page
- option to start game

- simple design overall
- good overall dimensions



This is the screen that loads up after the play button is pressed. The screen displays the health on the top left,the player on the centre of the screen and the score on the top right of the screen. All this proves to be working in the intended way.

<u>criteria met within screenshot:</u>

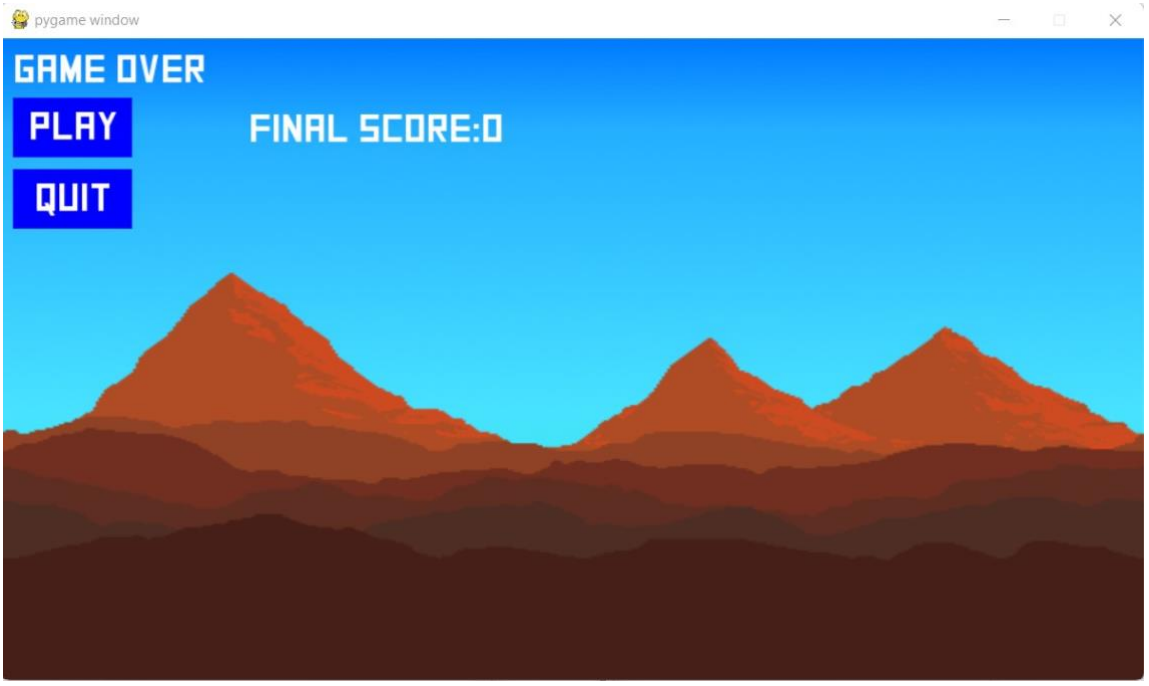
- character health bar
- score system
- simple overall design



This is a screenshot showing the enemies moving towards the player. Some of the enemies have dealt damage as the game went on, shown by the reduction in the amount of hearts on the top left and so that functionality clearly works.

criteria met within screenshot:

- the character can move
- has suitable movement and attack animations
- the enemies look suitable
- enemies can attack and deal damage



This is the screen that comes up when the player's health reaches 0. The screen allows the player to decide whether or not to play again or quit. the final score is displayed and so this means that the intended function works.

success criteria met:

- game over screen
- if player dies the screen is displayed
- option to play again once the player is dead

final code:

gameloop/main.py

```
import pygame
import sys
from pygame.locals import *

import enemy
import player

from player import button, Player, HealthBar

pygame.init()

FPS = 100
FPS_CLOCK = pygame.time.Clock()
font = pygame.font.Font("square.ttf", 32)
# Global variables
SCREEN_WIDTH = 960
SCREEN_HEIGHT = 540

# Setting up color objects
BLUE = (0, 0, 255)
RED = (255, 0, 0)
GREEN = (0, 255, 0)
BLACK = (0, 0, 0)
WHITE = (255, 255, 255)

# Set up pygame display object
displaysurface = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))

score_value = 0
textX = 800
```

```
textY = 10

def show_score(x, y):
    score = font.render("score :" + str(score_value), True , WHITE)
    displaysurface.blit(score,(x,y))

class Background(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.bgimage = pygame.image.load("game_background_1.png")
        self.bgY = 0
        self.bgX = 0

    def render(self):
        displaysurface.blit(self.bgimage, (self.bgX, self.bgY))

# Initialise background and health objects
background = Background()

Health = HealthBar()

class game:
    def __init__(self):
        self.running = True
        self.font = pygame.font.Font("square.ttf", 32)
        self.screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
        self.intro_background = pygame.image.load("finalDay.png")

    def intro_screen(self):
        # display variable controls the menu game loop
```

```
    display = True
    title = self.font.render("PYTHON GAME", True, WHITE)
    title_rect = title.get_rect(x=10, y=10)
    play_button = button(10, 50, 100, 50, WHITE, BLUE, "play", 32)
    quit_button = button(10, 110, 100, 50, WHITE, BLUE, "quit", 32)
    while display:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                display = False
                self.running = False

        mouse_pos = pygame.mouse.get_pos()
        mouse_pressed = pygame.mouse.get_pressed()
        if play_button.is_pressed(mouse_pos, mouse_pressed):
            # Close the menu object and start the game
            display = False
        if quit_button.is_pressed(mouse_pos, mouse_pressed):
            pygame.quit()
            sys.exit()

        self.screen.blit(self.intro_background, (0, 0))
        self.screen.blit(title, title_rect)
        self.screen.blit(play_button.image, play_button.rect)
        self.screen.blit(quit_button.image, quit_button.rect)
        pygame.display.update()

def game_over_screen(self, player, Health):
    global score_value
    # display variable controls the menu game loop
    display = True
```

```
title = self.font.render("GAME OVER", True, WHITE)
title_rect = title.get_rect(x=10, y=10)
play_button = button(10, 50, 100, 50, WHITE, BLUE, "play", 32)
quit_button = button(10, 110, 100, 50, WHITE, BLUE, "quit", 32)
score = self.font.render(" final Score:" + str(score_value), True, WHITE)
score_rect = score.get_rect(x=200, y=60)
while display:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            display = False
            self.running = False

    mouse_pos = pygame.mouse.get_pos()
    mouse_pressed = pygame.mouse.get_pressed()
    if play_button.is_pressed(mouse_pos, mouse_pressed):
        # Reset all key game variables
        score_value = 0
        player.reset_health(Health)
        display = False
    if quit_button.is_pressed(mouse_pos, mouse_pressed):
        pygame.quit()
        sys.exit()

    self.screen.blit(self.intro_background, (0, 0))
    self.screen.blit(title, title_rect)
    self.screen.blit(play_button.image, play_button.rect)
    self.screen.blit(quit_button.image, quit_button.rect)
    self.screen.blit(score, score_rect)
    pygame.display.update()
```

```
gameMenu = game()
gameMenu.intro_screen()
hit_cooldown = pygame.USEREVENT + 1

player = player.Player()

enemyGroup = pygame.sprite.Group()


ENEMY_SPAWN = pygame.USEREVENT + 2
pygame.time.set_timer(ENEMY_SPAWN, 5000)


# Game loop
while True:
    for event in pygame.event.get():
        if event.type == QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.MOUSEBUTTONDOWN:
             if player.attacking == False:
                 player.attack()
                 player.attacking = True
        if event.type == hit_cooldown:
            player.cooldown = False
            pygame.time.set_timer(hit_cooldown, 0)
        if event.type == ENEMY_SPAWN:
            enemyGroup.add(enemy.Enemy())

    background.render()
```

```
# Move enemies
for enemySprite in enemyGroup:
    # enemyGroup is a spritegroup, need to validate enemySprite as type enemy
    if isinstance(enemySprite, enemy.Enemy):
        enemySprite.update()
        enemySprite.move(player.rect.center)
        enemySprite.draw(displaysurface)

# Move player
player.update()
if player.attacking == True:
    player.attack()
else:
    player.move()
player.draw(displaysurface)

# Player and enemy collision behaviour
if pygame.sprite.spritecollideany(player, enemyGroup):
    if player.attacking == True:
        print("attack")

        for enemySprite in enemyGroup:
            if isinstance(enemySprite, enemy.Enemy):
                enemySprite.enemy_hit(player)
                score_value += 1
    else:
        player.player_hit(hit_cooldown,Health)
        if player.health == 0:
            gameMenu.game_over_screen(player, Health)
```

```
            gameMenu.game_over_screen(player, Health)
    if player.health > 0:
        displaysurface.blit(player.image, player.rect)
Health.render()


FPS_CLOCK.tick(FPS)
show_score(textX, textY)
pygame.display.update()
```

[player.py final code:]

```
import pygame
import sys
from pygame.locals import *
SCREEN_WIDTH = 960
SCREEN_HEIGHT = 540

displaysurface = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
# Set player animations
run_ani_R = [pygame.image.load("player/Movement_Animations/Player_Sprite_R.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite2_R.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite3_R.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite4_R.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite5_R.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite6_R.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite_R.png")]

run_ani_L = [pygame.image.load("player/Movement_Animations/Player_Sprite_L.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite2_L.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite3_L.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite4_L.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite5_L.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite6_L.png"),
             pygame.image.load("player/Movement_Animations/Player_Sprite_L.png")]

# Attack animation for the RIGHT
attack_ani_R = [pygame.image.load("player/Attack_Animations/Player_Attack_R.png"), pygame.image.load("player/Attack_Animations/Player_Attack_
                pygame.image.load("player/Attack_Animations/Player_Attack2_R.png"), pygame.image.load("player/Attack_Animations/Player_Attack
                pygame.image.load("player/Attack_Animations/Player_Attack3_R.png"), pygame.image.load("player/Attack_Animations/Player_Attack
                pygame.image.load("player/Attack_Animations/Player_Attack4_R.png"), pygame.image.load("player/Attack_Animations/Player_Attack
                pygame.image.load("player/Attack_Animations/Player_Attack5_R.png"), pygame.image.load("player/Attack_Animations/Player_Attack
```

```
                pygame.image.load("player/Attack_Animations/Player_Attack3_R.png"), pygame.image.load("player/Attack_Animati
                pygame.image.load("player/Attack_Animations/Player_Attack4_R.png"), pygame.image.load("player/Attack_Animations/Player_Attac
                pygame.image.load("player/Attack_Animations/Player_Attack5_R.png"), pygame.image.load("player/Attack_Animations/Player_Attack
                pygame.image.load("player/Attack_Animations/Player_Attack_R.png")]

# Attack animation for the LEFT
attack_ani_L = [pygame.image.load("player/Attack_Animations/Player_Attack_L.png"), pygame.image.load("player/Attack_Animations/Player_Attack_
                pygame.image.load("player/Attack_Animations/Player_Attack2_L.png"), pygame.image.load("player/Attack_Animations/Player_Attack
                pygame.image.load("player/Attack_Animations/Player_Attack3_L.png"), pygame.image.load("player/Attack_Animations/Player_Attack
                pygame.image.load("player/Attack_Animations/Player_Attack4_L.png"), pygame.image.load("player/Attack_Animations/Player_Attack
                pygame.image.load("player/Attack_Animations/Player_Attack5_L.png"), pygame.image.load("player/Attack_Animations/Player_Attack
                pygame.image.load("player/Attack_Animations/Player_Attack_L.png")]

health_ani = [pygame.image.load("health/heart0.png"), pygame.image.load("health/heart.png"),
              pygame.image.load("health/heart2.png"), pygame.image.load("health/heart3.png"),
              pygame.image.load("health/heart4.png"), pygame.image.load("health/heart5.png")]
```

```
class Player(pygame.sprite.Sprite):
    def __init__(self):
        # Inherit from the pygame.sprite.Sprite superclass
        super().__init__()
        self.image = run_ani_R[0]
        self.rect = self.image.get_rect()
        self.rect.center = (SCREEN_WIDTH / 2, SCREEN_HEIGHT / 2)
        # Combat

        self.attacking = False
        self.cooldown = False
        self.attack_frame = 0
        # Set the default variables
        self.health = 5
        self.score = 0
        self.direction = "RIGHT"

        # Set up animation variables
        # self.running = False
        self.move_frame = 0
        self.xpos = (0,0)
        self.ypos = (0,0)
    def update(self):
        # Return to base frame if at end of movement sequence
        if self.move_frame > 6:
            self.move_frame = 0
            return
    def move(self):
        # Change condition values to fit in the background image setting boundary for the player to move
```

```
pressed_keys = pygame.key.get_pressed()
if self.rect.left > 0:
    if pressed_keys[K_a]:
        self.image = run_ani_L[self.move_frame]
        self.rect.move_ip(-5, 0)
        self.direction = "LEFT"
if self.rect.right < SCREEN_WIDTH:
    if pressed_keys[K_d]:
        self.image = run_ani_R[self.move_frame]
        self.rect.move_ip(5, 0)
        self.direction = "RIGHT"
if self.rect.top > 40:
    if pressed_keys[K_w]:
        self.rect.move_ip(0, -5)
        self.direction = "UP"
if self.rect.bottom < 485:
    if pressed_keys[K_s]:
        self.rect.move_ip(0, 5)
        self.direction = "DOWN"

if pressed_keys[K_s] == False and pressed_keys[K_w] == False and pressed_keys[K_d] == False and pressed_keys[
    K_a] == False:
    self.move_frame = 0
    if self.direction == "LEFT":
        self.image = run_ani_L[self.move_frame]
    else:
        self.image = run_ani_R[self.move_frame]
else:
    self.move_frame += 1
```

```
def attack(self):
    # If attack frame has reached end of sequence, return to base frame
    if self.attack_frame > 10:
        self.attack_frame = 0
        self.attacking = False

    # Check direction for correct animation to display
    if self.direction == "RIGHT":
        self.image = attack_ani_R[self.attack_frame]
    elif self.direction == "LEFT":

        self.image = attack_ani_L[self.attack_frame]
 # Update the current attack frame
    self.attack_frame += 1

def draw(self, surface):
    surface.blit(self.image, self.rect)

def correction(self):
    # Function is used to correct an error
    # with character position on left attack frames
    if self.attack_frame == 1:
        self.xpos -= 20
    if self.attack_frame == 10:
        self.ypos += 20

def player_hit(self,hit_cooldown,Health):
    if self.cooldown == False:
        self.cooldown = True  # Enable the cooldown
        pygame.time.set_timer(hit_cooldown, 2000)  # Resets cooldown in 1 second
```

```
            print("hit")
            self.health -= 1
            Health.image = health_ani[self.health]

    def reset_health(self,Health):
        self.health = 5
        Health.image = health_ani[self.health]

class button:
    def __init__(self, x, y, width, height, fg, bg, content, fontsize):
        self.font = pygame.font.Font("square.ttf", fontsize)
        self.content = content

        self.x = x
        self.y = y
        self.width = width
        self.height = height
        self.fg = fg
        self.bg = bg

        self.image = pygame.Surface((self.width, self.height))
        self.image.fill(self.bg)
        self.rect = self.image.get_rect()

        self.rect.x = self.x
        self.rect.y = self.y
        self.text = self.font.render(self.content, True, self.fg)
        self.text_rect = self.text.get_rect(center=(self.width / 2, self.height / 2))
        self.image.blit(self.text, self.text_rect)
```

```
    def is_pressed(self, pos, pressed):
        if self.rect.collidepoint(pos):
            if pressed[0]:
                return True
            return False
        return False


class HealthBar(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.image.load("health/heart5.png")

    def render(self):
        displaysurface.blit(self.image, (10, 10))
```

enemy.py final code:

```
import random
from pygame.locals import *
import pygame

font = pygame.font.Font

SCREEN_WIDTH = 960
SCREEN_HEIGHT = 540
WHITE = (255, 255, 255)

displaysurface = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
score_value = 0




class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        # Inherit from the pygame.sprite.Sprite superclass
        super().__init__()
        self.image = pygame.image.load("2_enemies_idle_001.png")  # Find image for enemy
        self.rect = self.image.get_rect()
        xPos = random.randint(0, 1)
        if xPos == 0:
            xPos = 0
        else:
            xPos = SCREEN_WIDTH
        yPos = random.randint(100, SCREEN_HEIGHT - 100)
```

```
        self.rect.center = (xPos, yPos)

        # Combat
        self.attacking = False
        self.attack_frame = 0

        # Set the default variables
        self.health = 5
        self.direction = "RIGHT"

        # Set up animation variables
        # self.running = False
        self.move_frame = 0
        self.xpos = (0, 0)
        self.ypos = (0, 0)

    def update(self):
        # Return to base frame if at end of movement sequence
        if self.move_frame > 6:
            self.move_frame = 0
            return

    def move(self, playerCoords):
        playerX = playerCoords[0]
        playerY = playerCoords[1]
        enemyX = self.rect.center[0]
        enemyY = self.rect.center[1]

        if self.rect.left > 0:
            if enemyX > playerX:
```

```
                # self.image = run_ani_L[self.move_frame]
                self.rect.move_ip(-1, 0)
                self.direction = "LEFT"
        if self.rect.right < SCREEN_WIDTH:
            if enemyX < playerX:
                # self.image = run_ani_R[self.move_frame]
                self.rect.move_ip(1, 0)
                self.direction = "RIGHT"
        if self.rect.top > 40:
            if enemyY > playerY:
                self.rect.move_ip(0, -1)
                # self.direction = "UP"
        if self.rect.bottom < 485:
            if enemyY < playerY:
                self.rect.move_ip(0, 1)
                # self.direction = "DOWN"

    def attack(self):
        # If attack frame has reached end of sequence, return to base frame
        if self.attack_frame > 10:
            self.attack_frame = 0
            self.attacking = False

        # Check direction for correct animation to display
        if self.direction == "RIGHT":
            self.image = attack_ani_R[self.attack_frame]
        elif self.direction == "LEFT":
            self.image = attack_ani_L[self.attack_frame]
        # Update the current attack frame
        self.attack_frame += 1
```

```
def draw(self, surface):
    surface.blit(self.image, self.rect)

def correction(self):
    # Function is used to correct an error
    # with character position on left attack frames
    if self.attack_frame == 1:
        self.xpos -= 20
    if self.attack_frame == 10:
        self.ypos += 20

def enemy_hit(self,player):
    if pygame.sprite.collide_rect(player,self):
        print("enemy hit")

        self.kill()
```

This is all the code of my solution, all split into three different scripts that get imported into the main.py script so that the game runs.

things i missed when creating the solution and how it can be done for future development:

While creating the solution there were a few aspects of the design/criteria I have missed out on. This is mainly due to the time constraints I have.had on the creation of my solution.

1. coin system within the game:

The first thing I have not been able to do was the coin based system of my game. The coins within the game were meant to be a simple side aspect of the game that could keep the user occupied alongside killing the enemies. I did not create the coin count or the coin spawning function as I felt that it was more important to do other things such as player and enemy functions. This meant I could not add in the coins to my game. in the future to add the coins i could make a class for the coins and potentially add incremental spawns

2. setting menu:

In the creation of my program I did not have time to create the settings menu and felt it would be best to sort out the play and quit buttons first.

3. adjustable sound:

I did not create a method of adjusting the sound in my game. This is because I had no time to search and add music to my game in the first place and so there was no point in developing an adjustable music method for my code.

4. pause menu:

I did not have enough time to create a pause menu for the game and for future development the creation of a pause menu will involve suspending the game loop and working with that.

5. changing controls

In the creation of the game, after testing it around, I felt there was not really a need to add a changing controls option. this is because i was running low on time and felt that my game was fairly simple anyway so it wouldn't be necessary to emphasise my time on creating adjustable keys

6. information screen

I did not have enough time to create an information screen for my game.

usability features of my solution:

In my solution there are not a lot of usability features. This is because due to a lack of time I could not implement many of them. For instance the information screen was meant to act as a tutorial for the new users but as of now there is no existence of one.

The usability features I have included are that the game overall is very simple and easy to understand so new users will have no problem getting used to the game. I also have used aspects of memorability in my game such as a red health bar with hearts or a green zombie mob with a weapon to be the enemies. all these things will mean for a new user it will be easy to understand what's going on.



This image shows the start screen of my game. The start screen has very bold and clear buttons with the purpose of the buttons being very easy to understand. This is good for the users starting my application for the first time since they should most likely know what to do straight away when they start the game. the fact the writing is bold also distinguished to the user that its a clickable button.



This screenshot is used to show the memorable aspect and familiarity aspect of my game. The arrows show the health bar and the enemies on my game screen. This is to demonstrate how even if the arrow wasn't there most users would be able to identify the purpose of the

hearts or the significance of a green zombie-like figure moving around the screen. This allows for a simple and easy to understand user experience within my game.

limitations of my solution:

There are many limitations to my solution. The first limitation of my solution is that the user will need a device with a python interpreter to run the code. This means that as of now my game only works on computers. My game is also fairly simple in design. This may be a good thing for the new users of my game but for experienced gamers or long time players of my game the whole genre may get boring and repetitive for the users.

Another potential limitation of my solution is that there is a lack of variety in the game. For instance there is only one type of player and one type of enemy. If I were to introduce some customizability into the game it would indeed help to improve the overall experience of the game. I could also try to introduce different maps so that the player does not have to look at the same map every time they play.

Another limitation of my game is that the enemies spawn at the same rate no matter how high the players score and so in the future I could try to implement a function that makes the game harder as the player progresses through the game.

Another limitation in my game is that the game's nature stays constant throughout the whole time a user plays it. This is because the only thing that is done is that the user attacks the same type of enemy constantly. This means it can get boring after some time as the game is very repetitive.

Another limitation to my game is that once the user is dead the score does not save and so all previous records and high scores get lost once the game ends.

Another limitation is that there is no pause button in my game. This means if the user needs to go away from the screen to do something else they will be forced to die and end the game. This can be a problem for people who are going for a really high score but have to do something at home for eg.

addressing the limitations:

In order to address most of these limitations it will involve me having to just simply develop more of the game and add in the factors that I said were limiting it. This means I would have to invest more time into the production of the game and this was not possible as of now. This was because of the time constraints I had been given otherwise the game would have had much more inside of it. I could potentially in the future get more developers to come and code the game which would definitely help improve the game and not have as many limitations since more people would be working on it at the same time.

improvements that can be made in the future:

In order to improve the game in the future to make it more fun there can be some more things that can be added.

- add more variety for the player by introducing skins or even more weapons
- add more levels
- potentially add in game objectives
- add levels that get increasingly difficult
- add all the things specific in the design section that was missed out due to time constraints
- if levels are introduced then a saving method should be added in order to save the level the player was on and add any recorded high scores **maintenance:**

As time goes on the game needs to be maintained and managed. The game will not need much maintenance as it will be able to run as long as the code is intact. it would be able to run on most devices which can run python. If any updates are needed to be made to the game it can be easily done as the classes and methods are fairly organised and through the use of three separate scripts in my creation of the game it makes it simple to make any changes to the code. overall there won't need to be much excess maintenance of the game since the game will not need a server or such.




```
# Set up pygame display object
displaysurface = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))

score_value = 0
textX = 800
textY = 10

def show_score(x, y):
    score = font.render("score :" + str(score_value), True, WHI
    displaysurface.blit(score, (x,y))

class Background(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.bgimage = pygame.image.load("game_background_1.png")
        self.bgY = 0
        self.bgX = 0

    def render(self):
        displaysurface.blit(self.bgimage, (self.bgX, self.bgY))

# Initialise background and health objects
```

,



Throughout my program I have added comments across all the classes and functions in order to give the developer an idea of what the purpose of each individual function is. The images shown above are only some of the comments in my code and they serve as an example of how the format will be like. The hope is that the comments will help aid development of my code in the future by other people who have never seen the code. The comments allow for the developer to quickly locate the section of code they are interested in and work on that however they please.